

Computer Vision 1

3-2: Optical Flow, Motion and Tracking

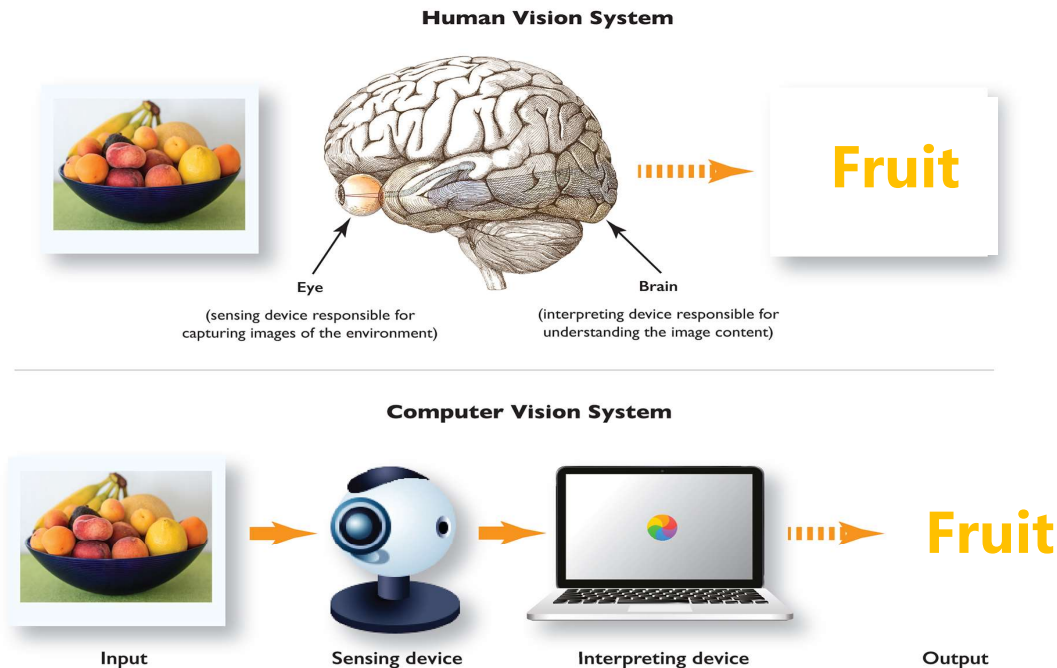
Master of AI – 2024/2025
University of Amsterdam



Arnoud Visser & Martin Oswald

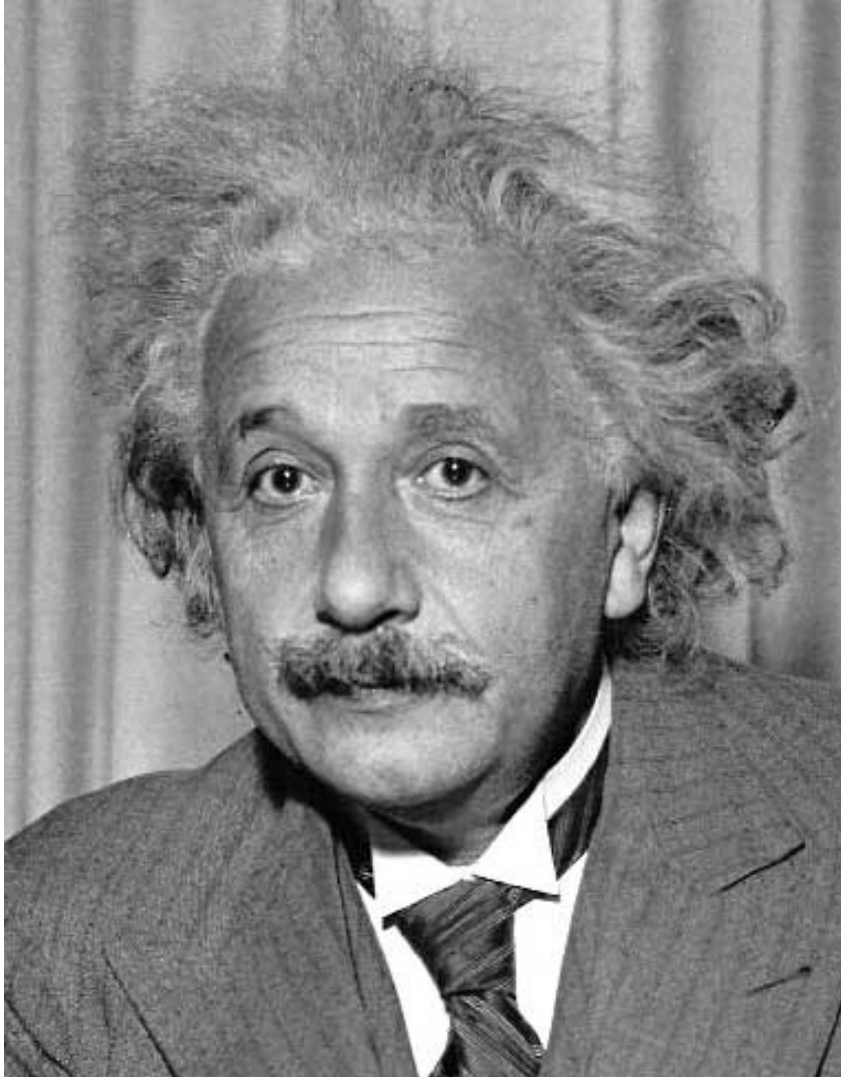
The computer vision problem

- Make a computer see and understand images



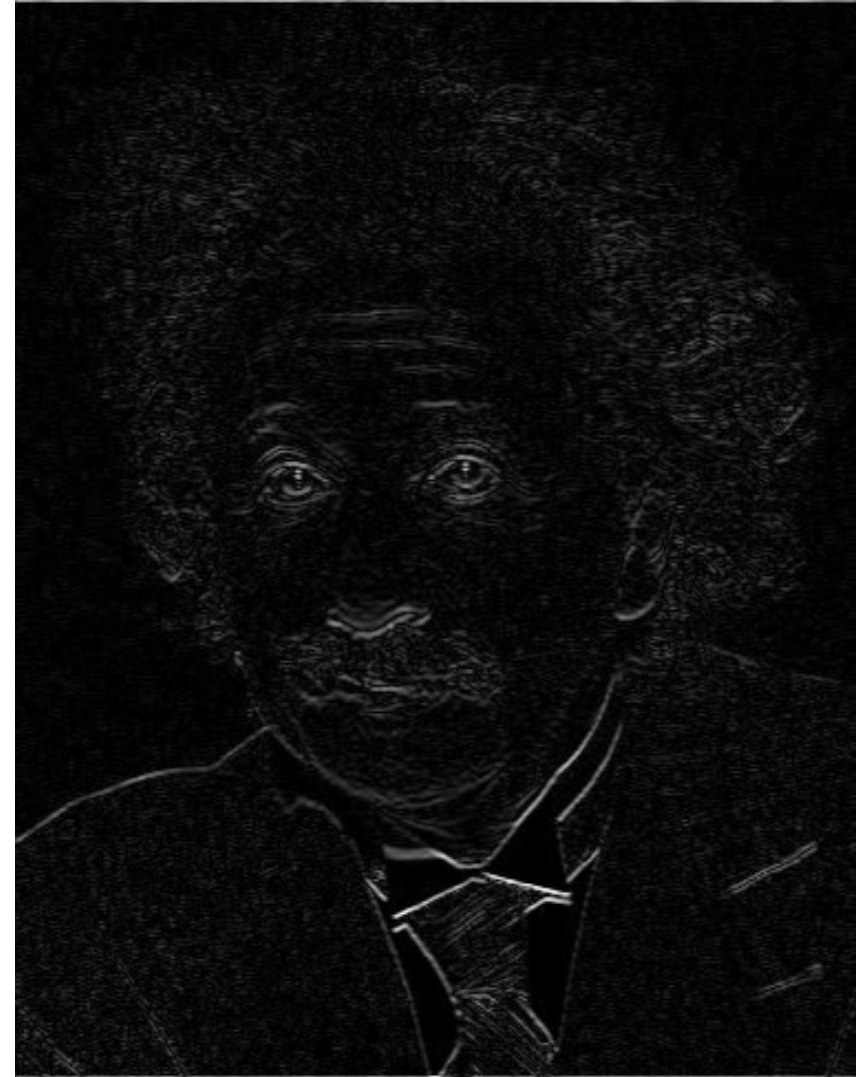
- We know it is physically possible –
we do it every day and effortlessly!

Derivative filters



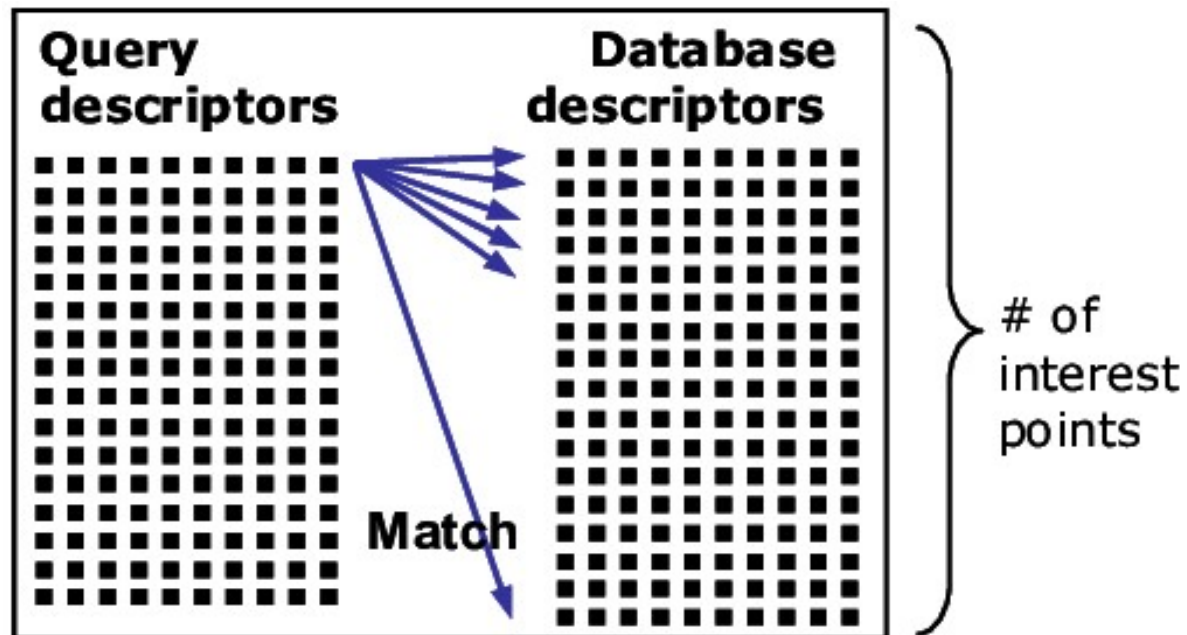
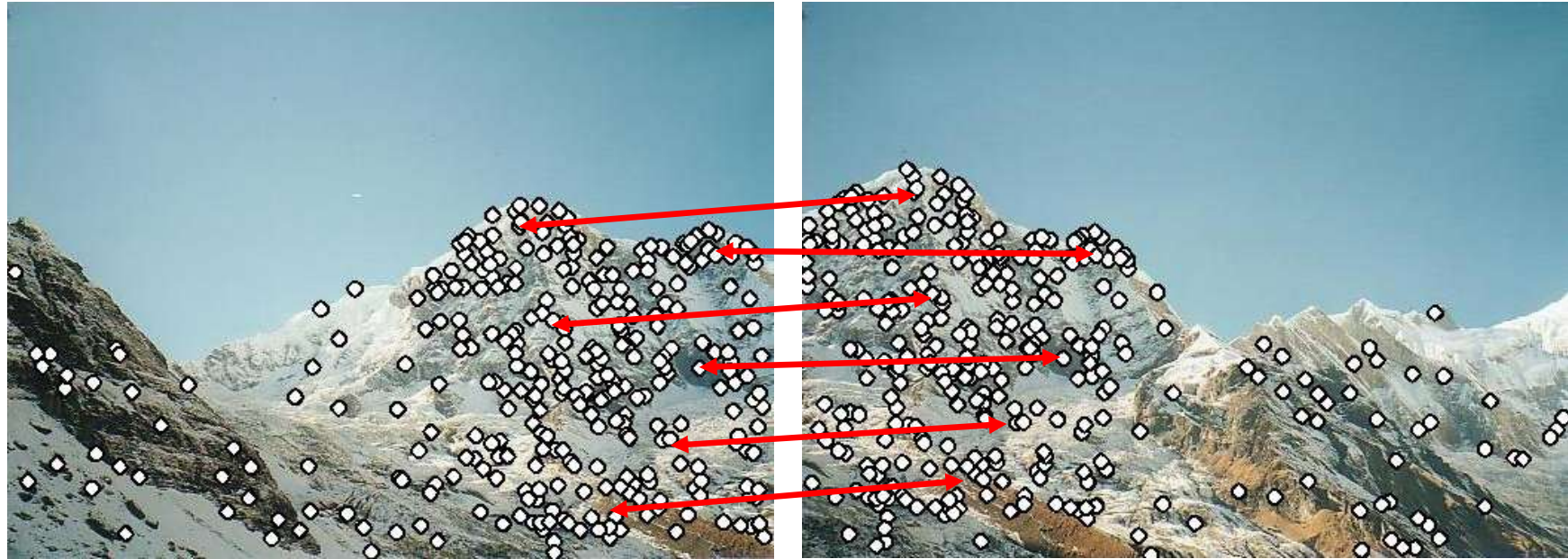
1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Match features: brutal force matcher

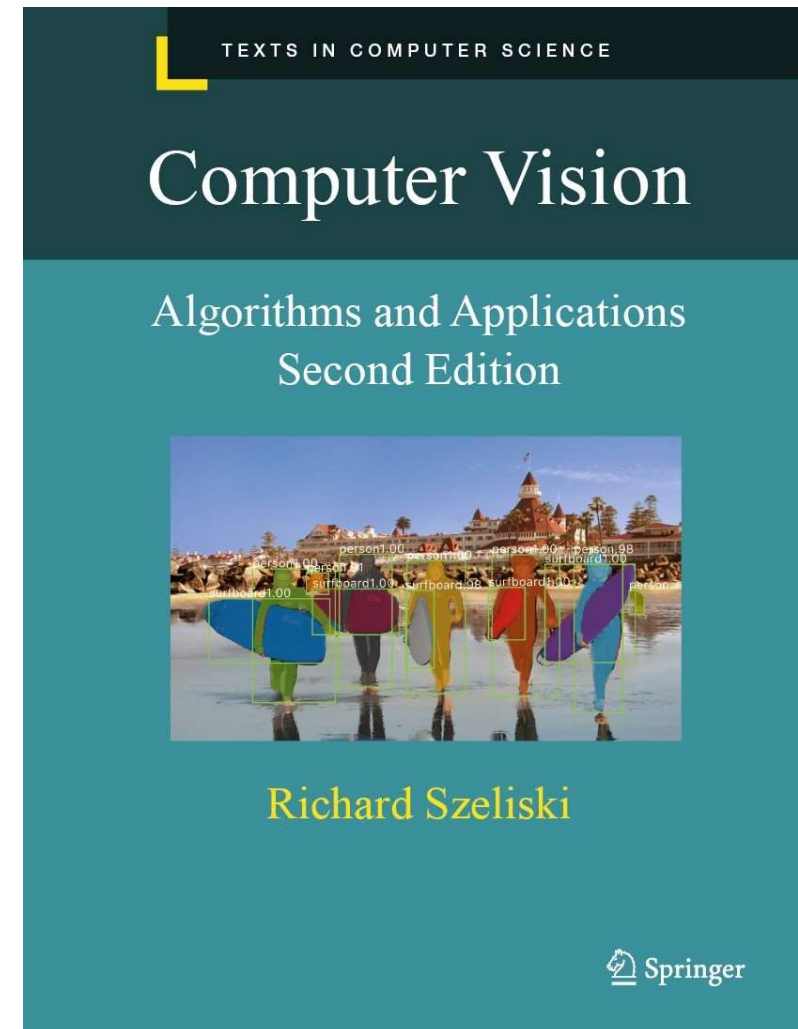


Outline

- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

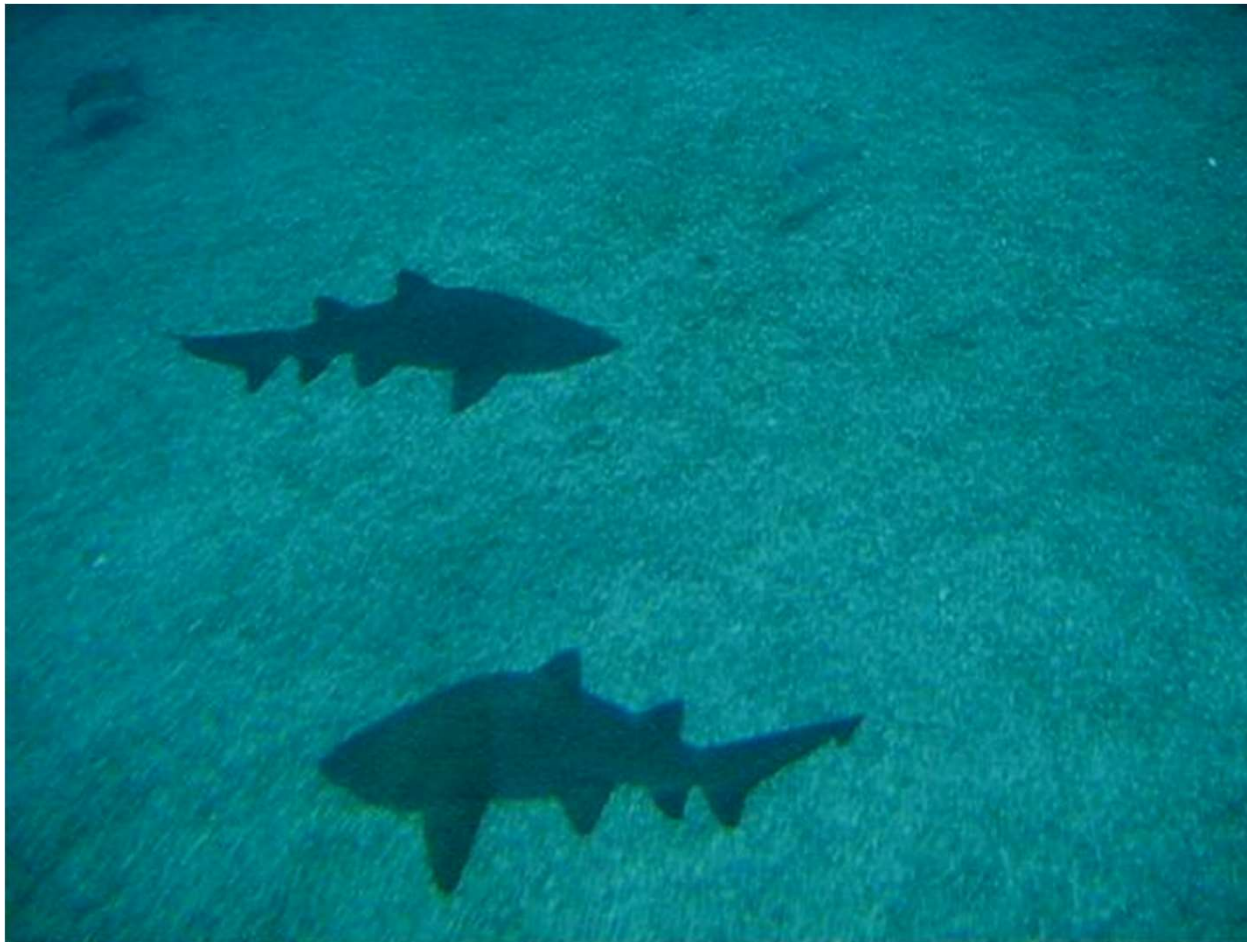
Textbook

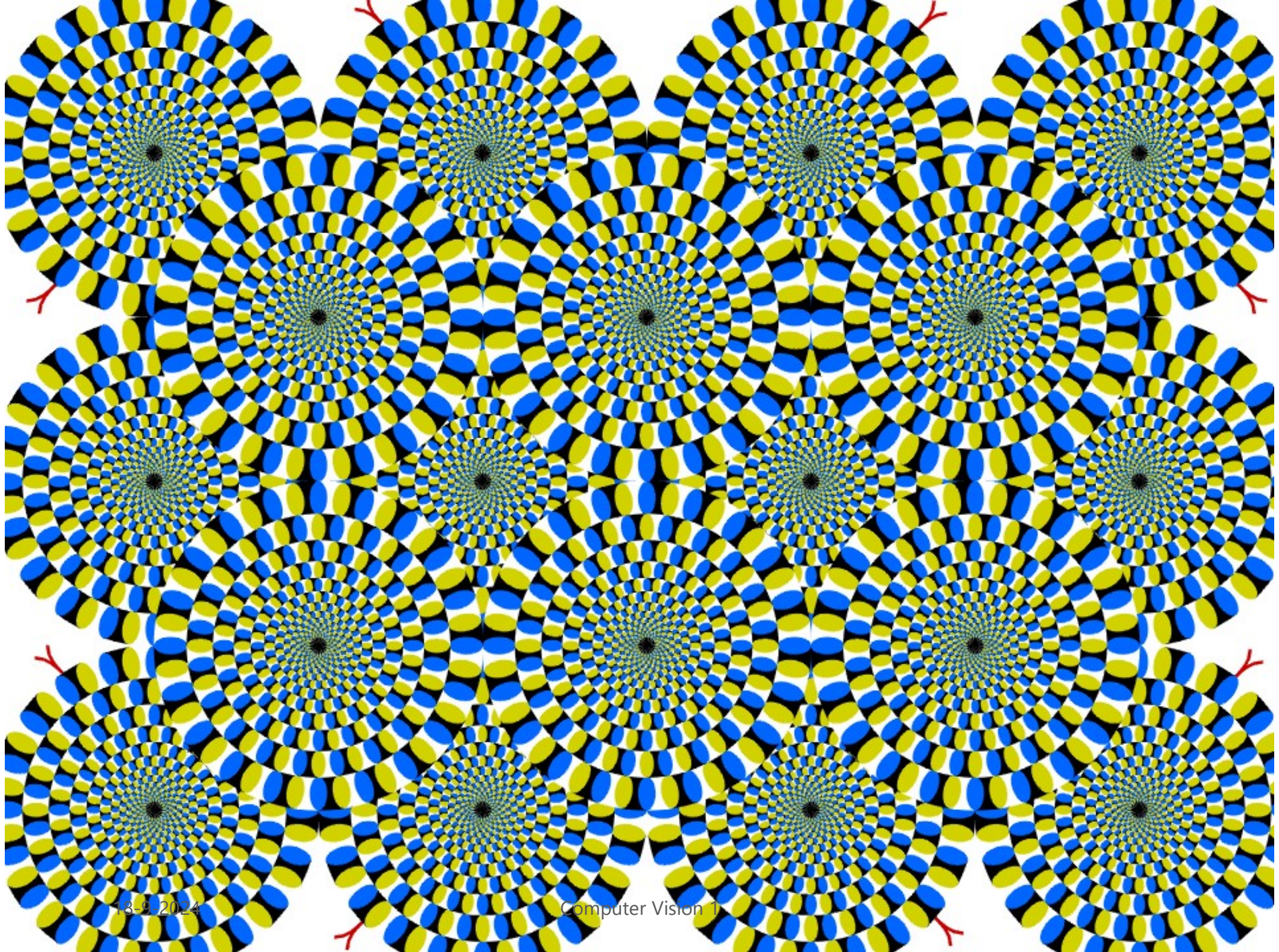
- 9.1.1
- 9.1.3
- 9.3 (9.3.1 optional)



We live in a moving world

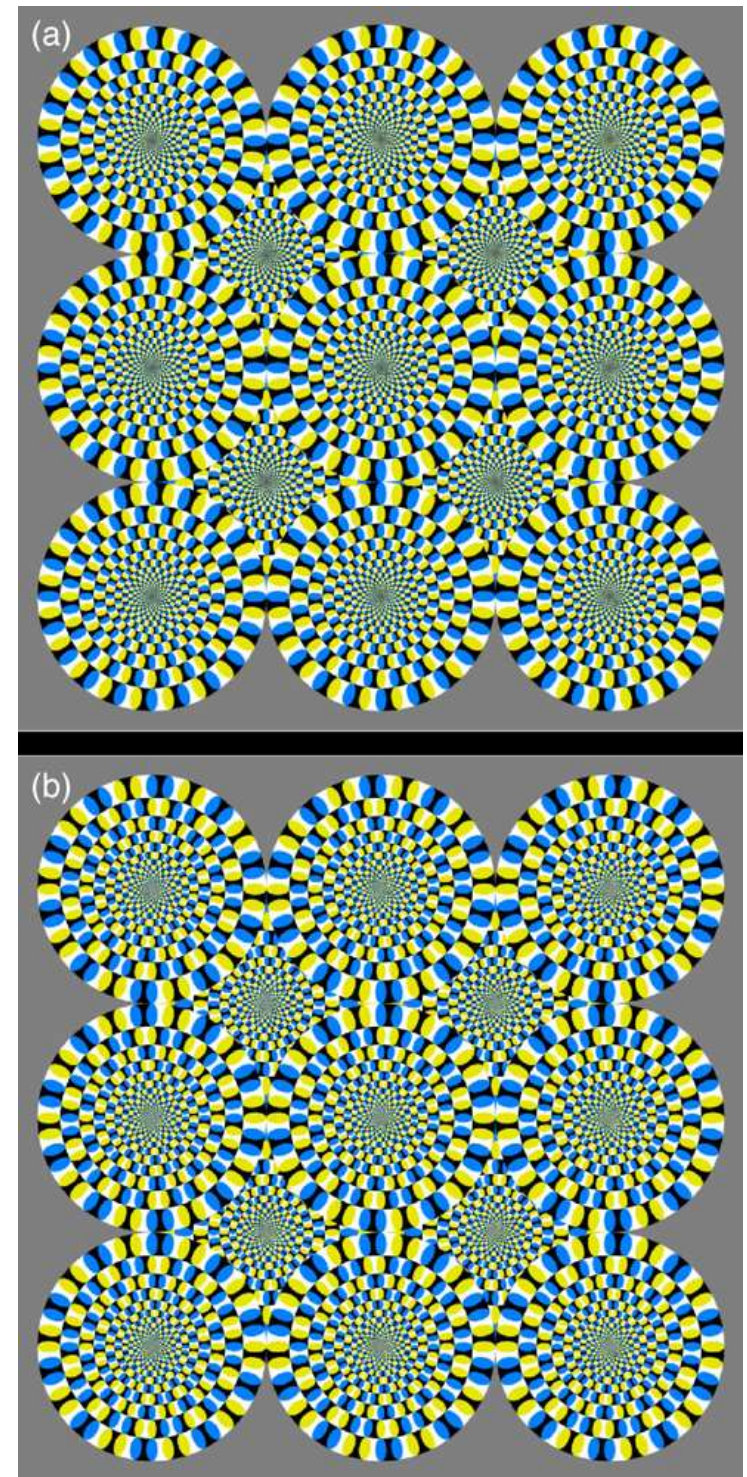
- Perceiving, understanding and predicting motion is an important part of our daily lives





How is this possible?

- The true mechanism is yet to be revealed
- fMRI data suggest that illusion is related to some component of eye movements
- We don't expect computer vision to "see" motion from these stimuli, yet



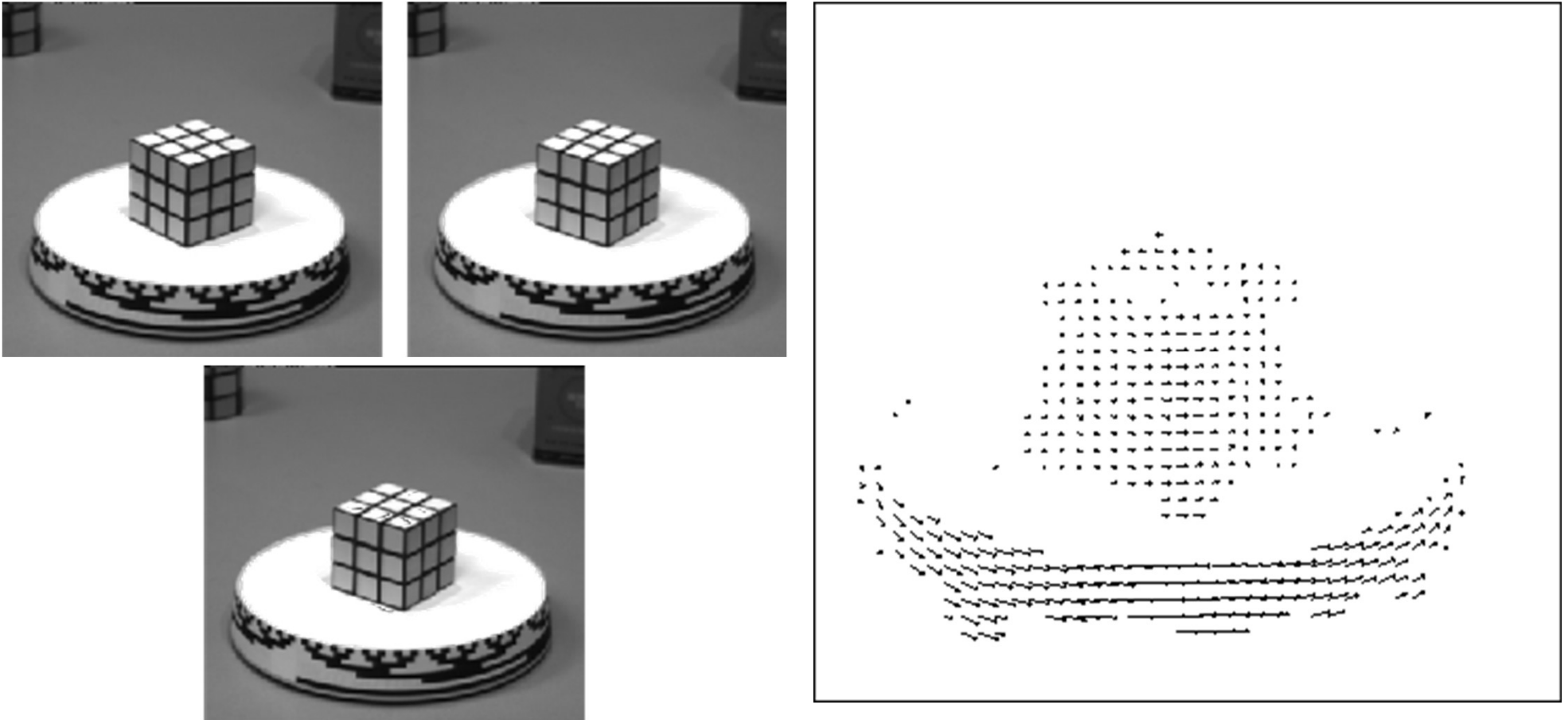
Motion and perceptual organization

- Even “impoverished” motion data can evoke a strong percept



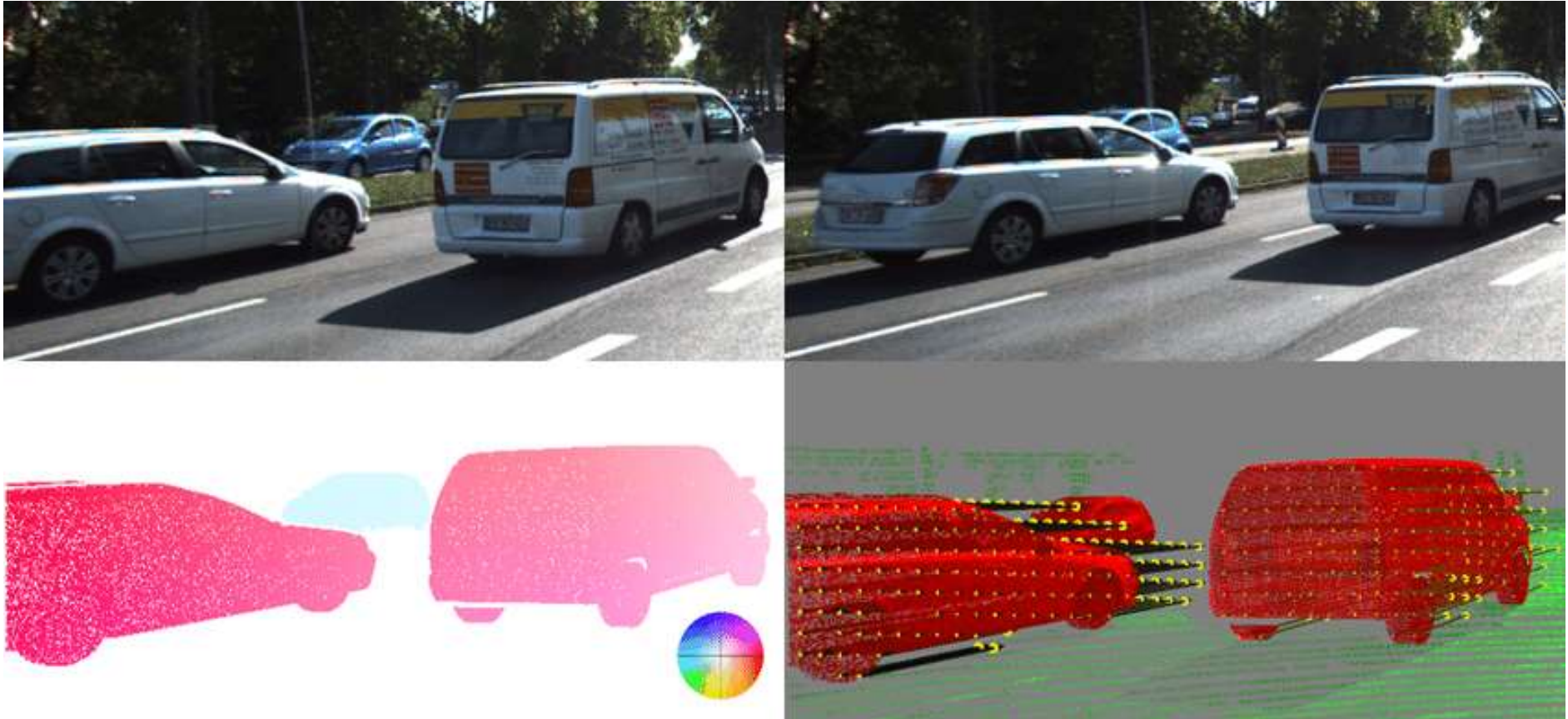
G. Johansson, “Visual Perception of Biological Motion and a Model For Its Analysis”, *Perception and Psychophysics* 14, 201-211, 1973.

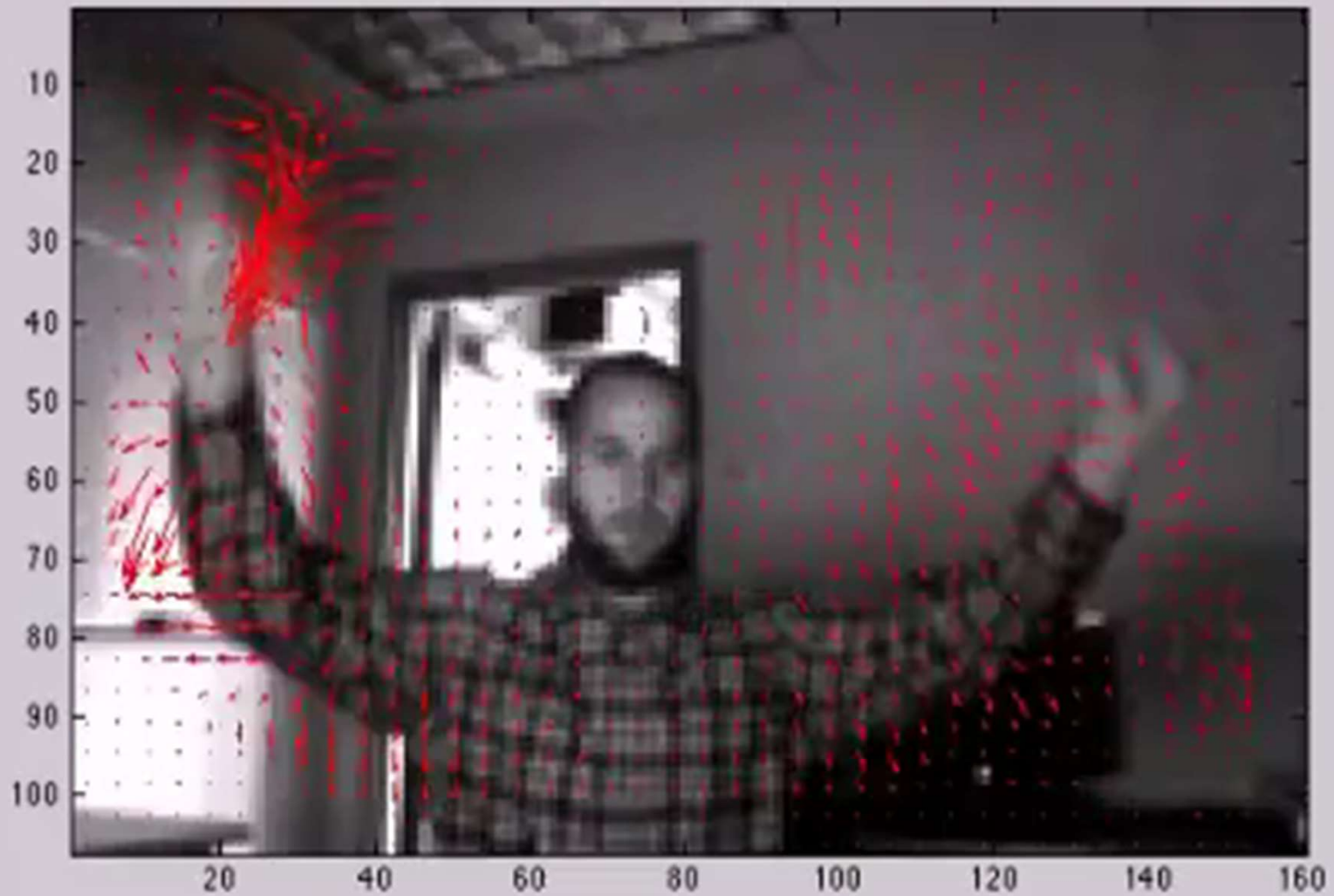
Motion estimation: Optical flow



Will start by estimating motion of each pixel separately
Then will consider motion of entire image

Vector Field of Per-pixel Motion







The cause of motion

- Three factors in imaging process
 - Light
 - Object
 - Camera
- Varying either of them causes motion
 - Static camera, moving objects (surveillance)
 - Moving camera, static scene (3D capture)
 - Moving camera, moving scene (sports, movie)
 - Static camera, moving objects, moving light (time lapse)



Why estimate motion?

- Lots of uses
 - Track object behavior
 - Correct for camera jitter (stabilization)
 - Align images (mosaics)
 - 3D shape reconstruction
 - Special effects

Motion scenarios (priors)



Static camera, moving scene



Moving camera, static scene



Moving camera, moving scene

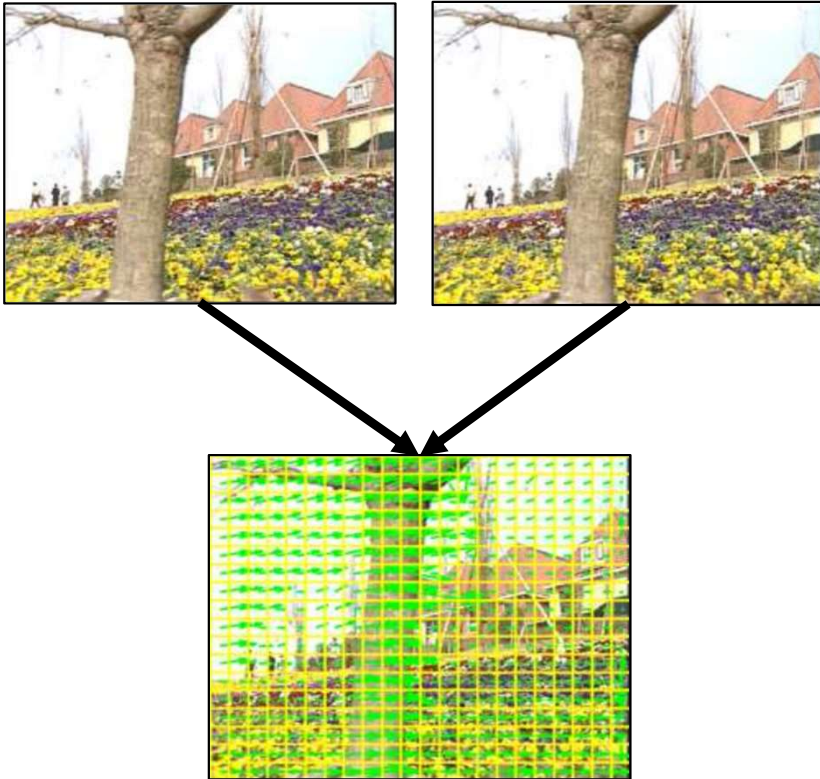


Static camera, moving scene, moving light

Why do we want Optical Flow?

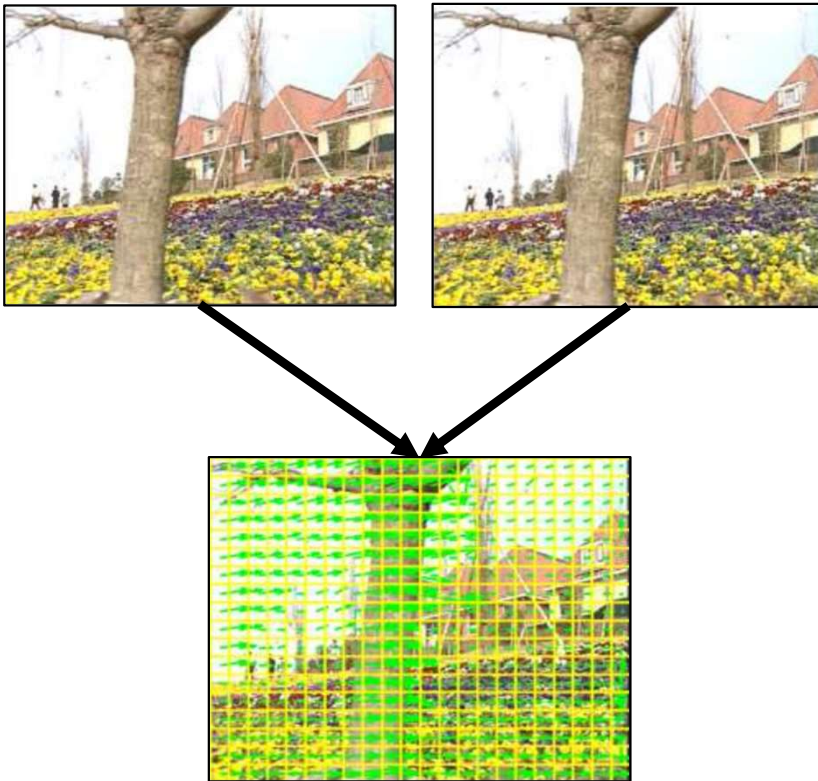
Why do we want Optical Flow?

Motion Estimation

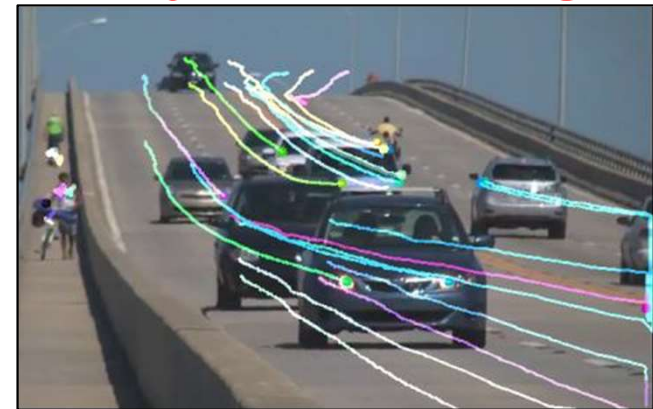


Why do we want Optical Flow?

Motion Estimation

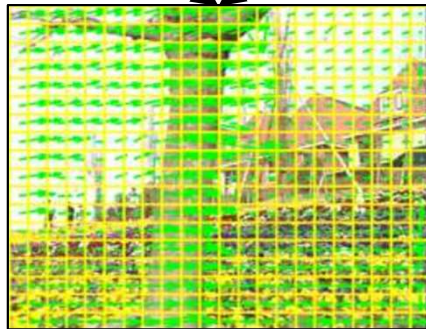


Object Tracking

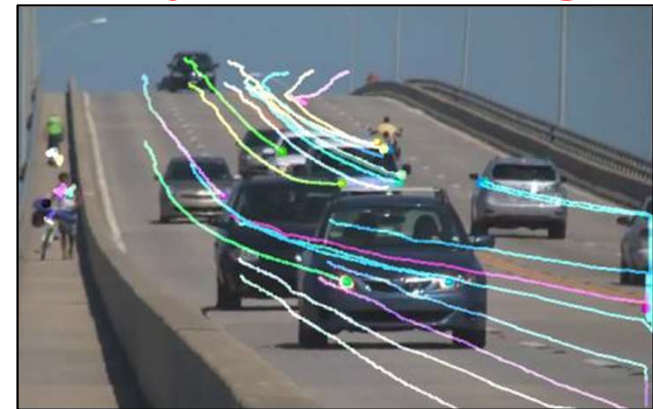


Why do we want Optical Flow?

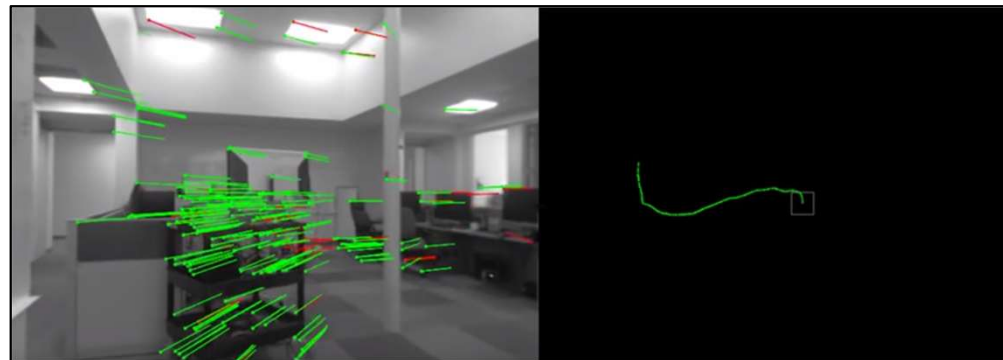
Motion Estimation



Object Tracking



Visual Odometry

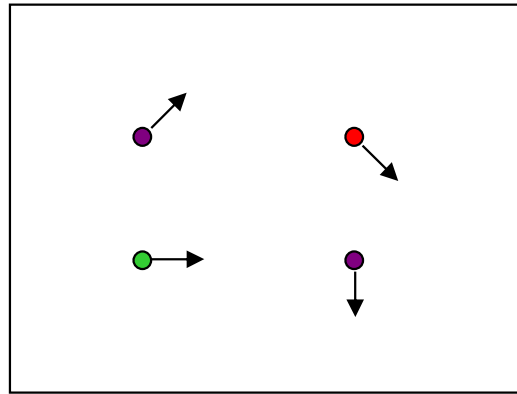


Estimating the position of a robot.

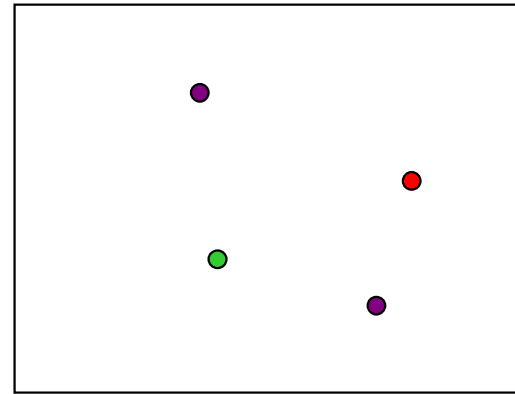
Outline

- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Iterative Lukas-Kanade Algorithm
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

Problem definition: optical flow



$H(x, y)$



$I(x, y)$

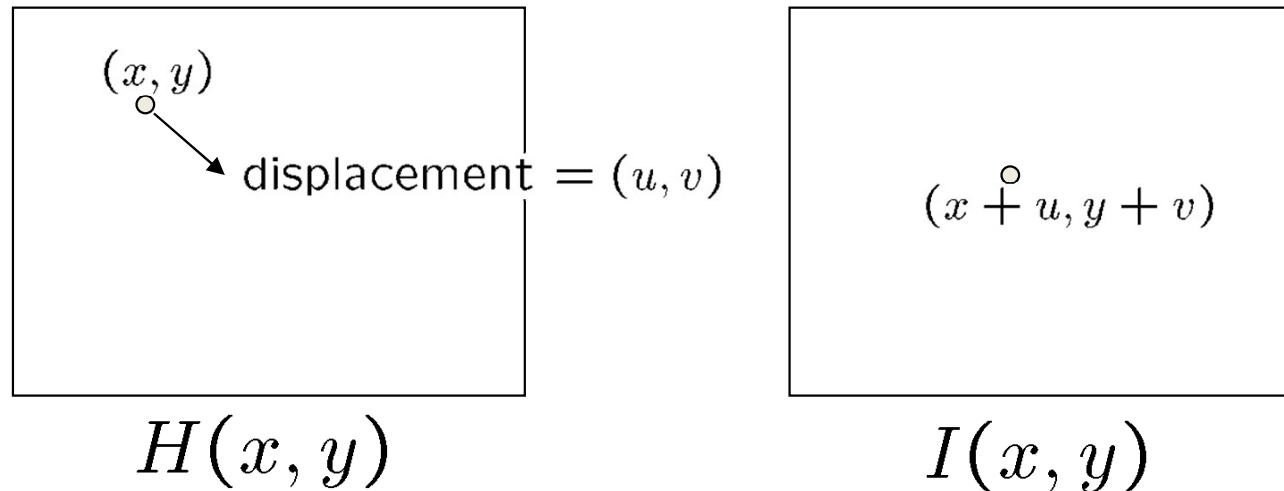
- How to estimate pixel motion from image H to image I?
 - Solve pixel correspondence problem
 - given a pixel in H, look for **nearby** pixels of the **same color** in I

Key assumptions

- **color constancy**: a point in H looks the same in I
 - For grayscale images, this is brightness constancy
- **small motion**: points do not move very far

This is called the optical flow problem

Optical flow constraints (grayscale images)



- Let's look at these constraints more closely

- brightness constancy: Q: what's the equation?

$$H(x, y) = I(x + u, y + v)$$

- small motion: (u and v are less than 1 pixel)
 - suppose we take the Taylor series expansion of I :

$$I(x + u, y + v) = I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \text{higher order terms}$$

$$\approx I(x, y) + \frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v$$

Optical flow equation

- Combining these two equations

$$0 = I(x + u, y + v) - H(x, y) \quad \text{shorthand: } I_x = \frac{\partial I}{\partial x}$$

$$\approx I(x, y) + I_x u + I_y v - H(x, y)$$

$$\approx (I(x, y) - H(x, y)) + I_x u + I_y v$$

$$\approx I_t + I_x u + I_y v$$

$$\approx I_t + \nabla I \cdot [u \ v]^T$$

Optical flow equation

$$0 \cong \sum_i [I_1(\mathbf{x}_i + \mathbf{u}_i) - I_0(\mathbf{x}_i)]^2$$

- Q: how many unknowns and equations per pixel?

2 unknowns, one equation

Intuitively, what does this constraint mean?

- The component of the flow in the gradient direction is determined
- The component of the flow parallel to an edge is unknown

This explains the Barber Pole illusion

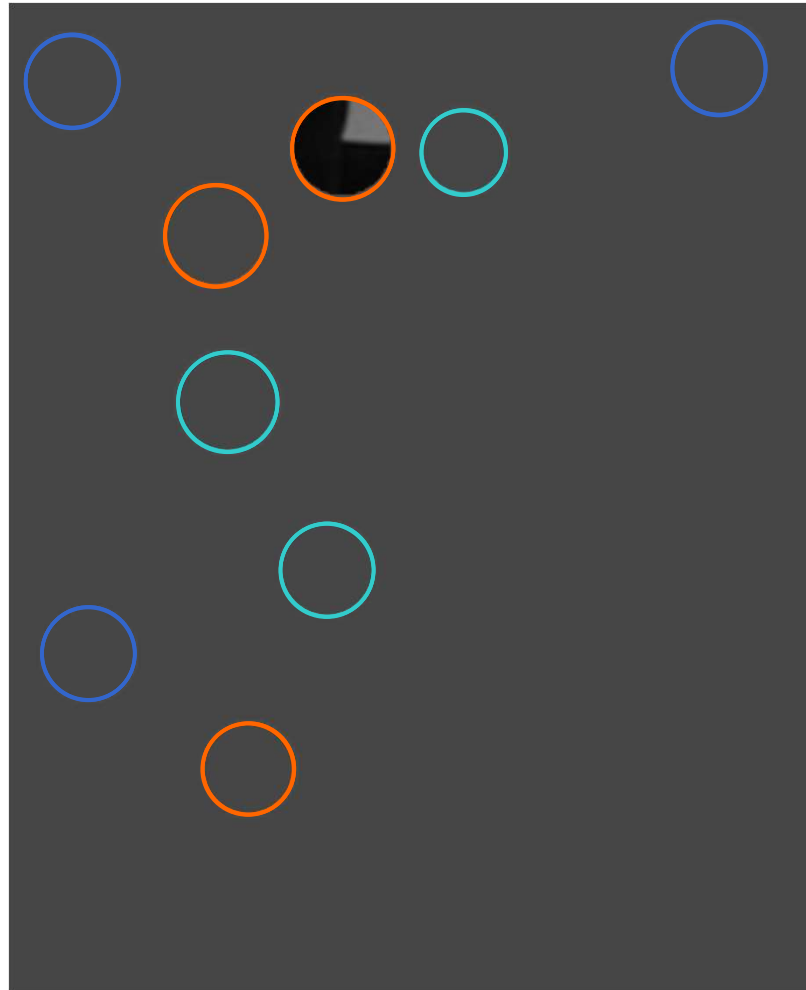
http://en.wikipedia.org/wiki/Barber's_pole



Outline

- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Iterative Lukas-Kanade Algorithm
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

Aperture problem

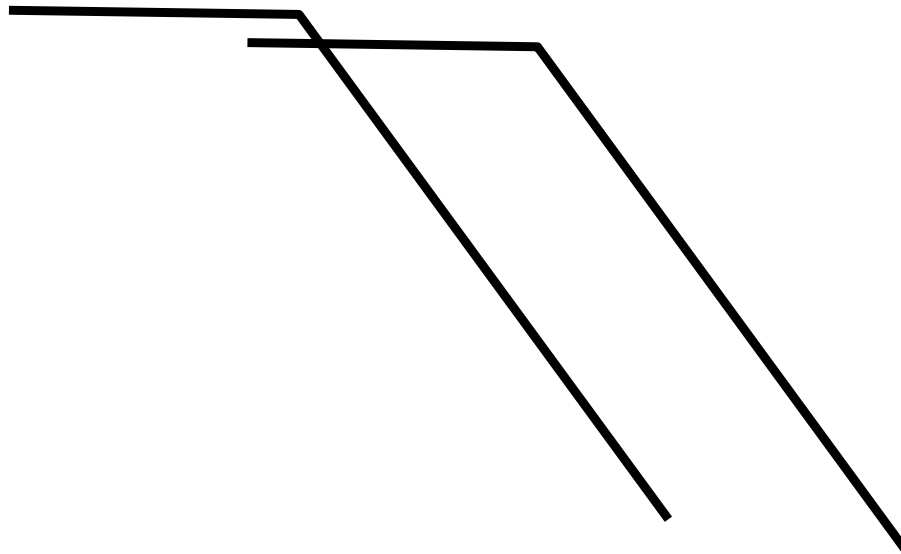


Corners

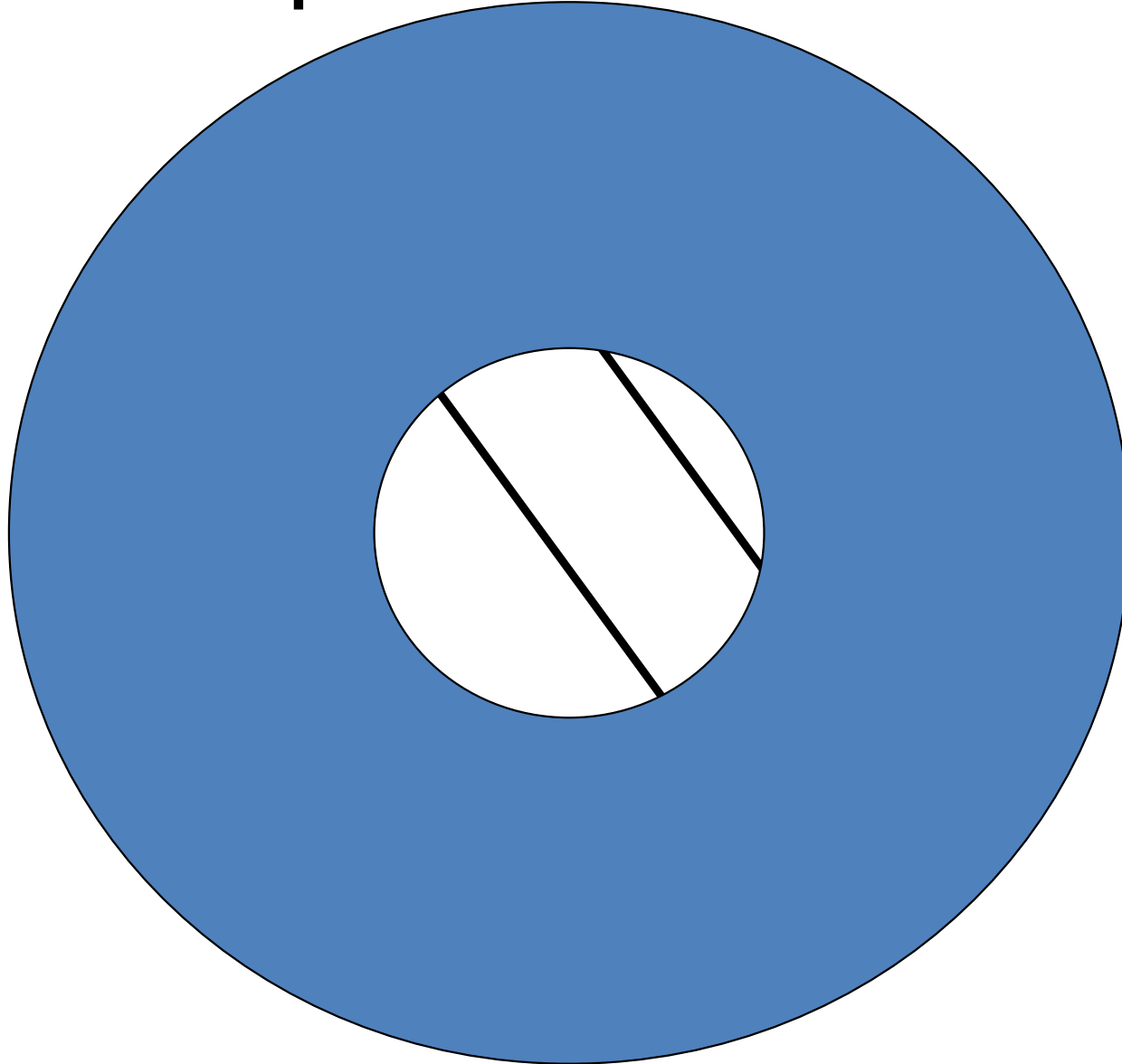
Lines

Flat regions

Aperture problem



Aperture problem



Outline

- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

Solving the aperture problem

- How to get more equations for a pixel?
 - Basic idea: impose additional constraints
 - most common is to assume that the flow field is smooth locally
 - one method: pretend the pixel's neighbors have the same (u,v)
 - If we use a **5x5 window**, that gives us 25 equations per pixel!

$$0 = I_t(\mathbf{p}_i) + \nabla I(\mathbf{p}_i) \cdot [u \ v]$$

$$\begin{array}{c} \left[\begin{array}{cc} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{array} \right] \begin{array}{c} \left[\begin{array}{c} u \\ v \end{array} \right] \end{array} = - \begin{array}{c} \left[\begin{array}{c} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{array} \right] \end{array} \\ \begin{array}{c} \mathbf{A} \\ 25 \times 2 \end{array} \qquad \begin{array}{c} \mathbf{d} \\ 2 \times 1 \end{array} \qquad \begin{array}{c} \mathbf{b} \\ 25 \times 1 \end{array} \end{array}$$

RGB version

$$0 = I_t(\mathbf{p}_i)[0, 1, 2] + \nabla I(\mathbf{p}_i)[0, 1, 2] \cdot [u \ v]$$

- How to get more equations for a pixel?
 - Basic idea: impose additional constraints
 - most common is to assume that the flow field is smooth locally
 - one method: pretend the pixel's neighbors have the same (u,v)
 - If we use a 5x5 window, that gives us 25*3 equations per pixel!

$$\begin{array}{ccc}
 \left[\begin{array}{cc} I_x(\mathbf{p}_1)[0] & I_y(\mathbf{p}_1)[0] \\ I_x(\mathbf{p}_1)[1] & I_y(\mathbf{p}_1)[1] \\ I_x(\mathbf{p}_1)[2] & I_y(\mathbf{p}_1)[2] \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25})[0] & I_y(\mathbf{p}_{25})[0] \\ I_x(\mathbf{p}_{25})[1] & I_y(\mathbf{p}_{25})[1] \\ I_x(\mathbf{p}_{25})[2] & I_y(\mathbf{p}_{25})[2] \end{array} \right] & \begin{bmatrix} u \\ v \end{bmatrix} = - & \left[\begin{array}{c} I_t(\mathbf{p}_1)[0] \\ I_t(\mathbf{p}_1)[1] \\ I_t(\mathbf{p}_1)[2] \\ \vdots \\ I_t(\mathbf{p}_{25})[0] \\ I_t(\mathbf{p}_{25})[1] \\ I_t(\mathbf{p}_{25})[2] \end{array} \right] \\
 \underset{75 \times 2}{A} & \underset{2 \times 1}{d} & \underset{75 \times 1}{b}
 \end{array}$$

Note that RGB is not enough to disambiguate
 because R, G & B are correlated
 Just provides better gradient

Lukas-Kanade flow

- Prob: we have more equations than unknowns

$$\begin{matrix} A & d = b \\ 25 \times 2 & 2 \times 1 \quad 25 \times 1 \end{matrix} \longrightarrow \text{minimize } \|Ad - b\|^2$$

Solution: solve least squares problem

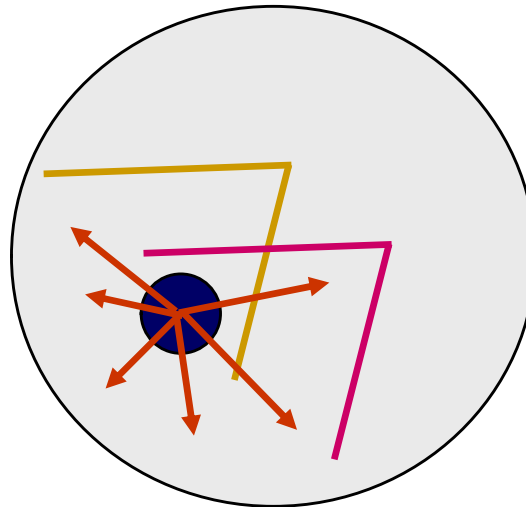
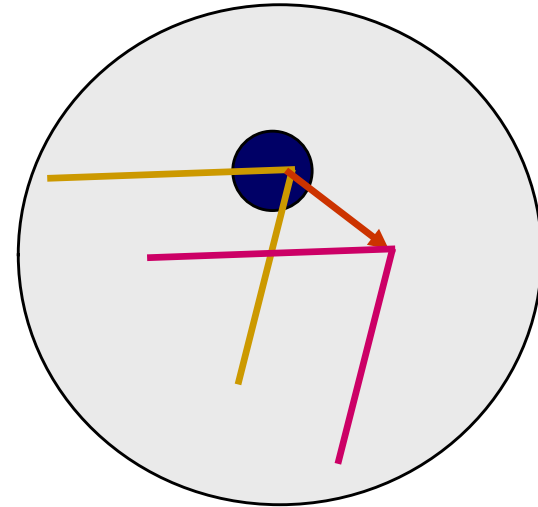
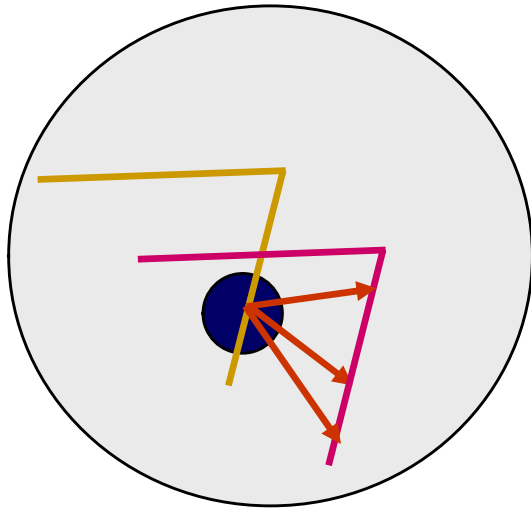
- minimum least squares solution given by solution (in d) of:

$$\begin{matrix} (A^T A) & d = A^T b \\ 2 \times 2 & 2 \times 1 \quad 2 \times 1 \end{matrix}$$

$$\begin{matrix} \begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} & \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix} \\ A^T A & A^T b \end{matrix}$$

- The summations are over all pixels in the K x K window
- This technique was first proposed by Lukas & Kanade (1981)

Local Patch Analysis



Conditions for solvability

- Optimal (u, v) satisfies Lucas-Kanade equation

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $A^T b$

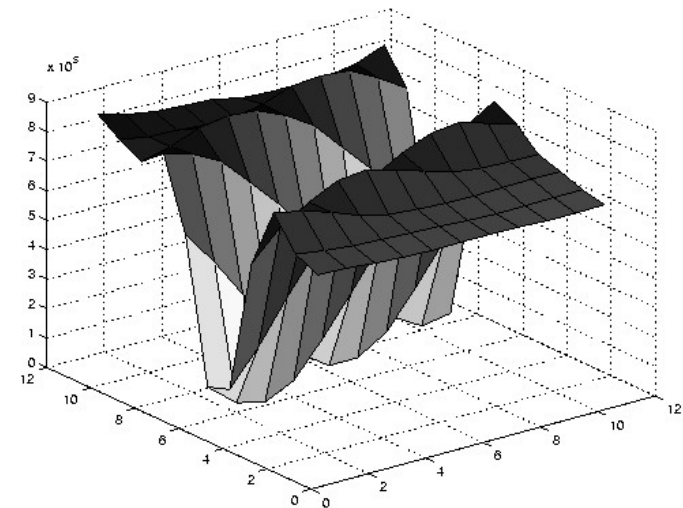
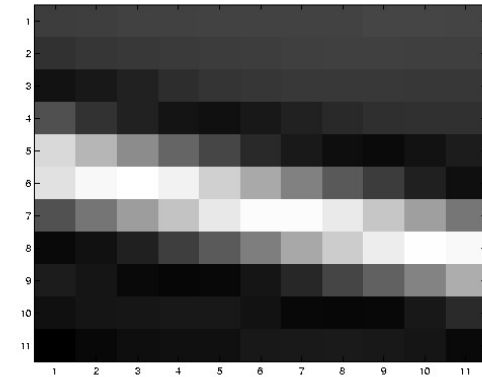
When is This Solvable?

- $A^T A$ should be invertible
- $A^T A$ should not be too small due to noise
 - eigenvalues λ_1 and λ_2 of $A^T A$ should not be too small
- $A^T A$ should be well-conditioned
 - λ_1 / λ_2 should not be too large (λ_1 = larger eigenvalue)

$A^T A$ is solvable when there is no aperture problem

$$A^T A = \begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} = \sum \begin{bmatrix} I_x \\ I_y \end{bmatrix} [I_x \ I_y] = \sum \nabla I (\nabla I)^T$$

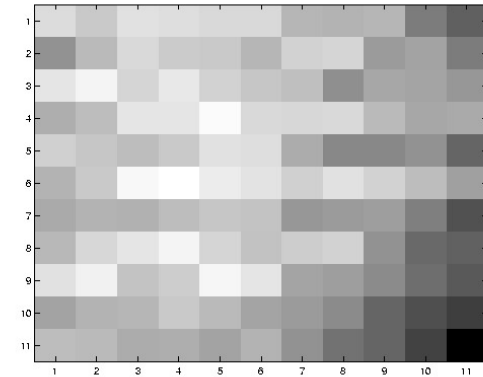
Edge



$$\sum \nabla I (\nabla I)^T$$

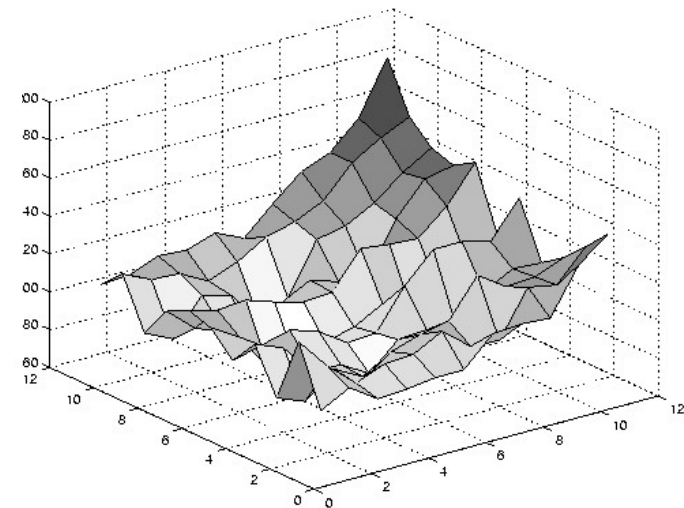
- large gradients, all the same
- large λ_1 , small λ_2

Low texture region

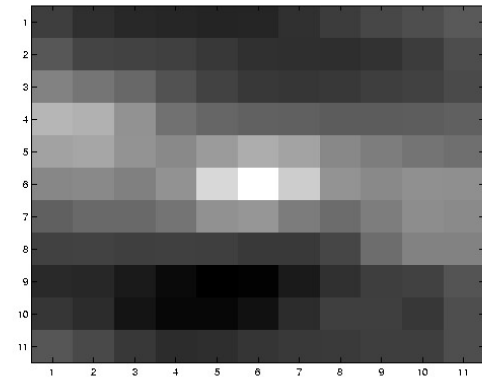


$$\sum \nabla I (\nabla I)^T$$

- gradients have small magnitude
- small λ_1 , small λ_2

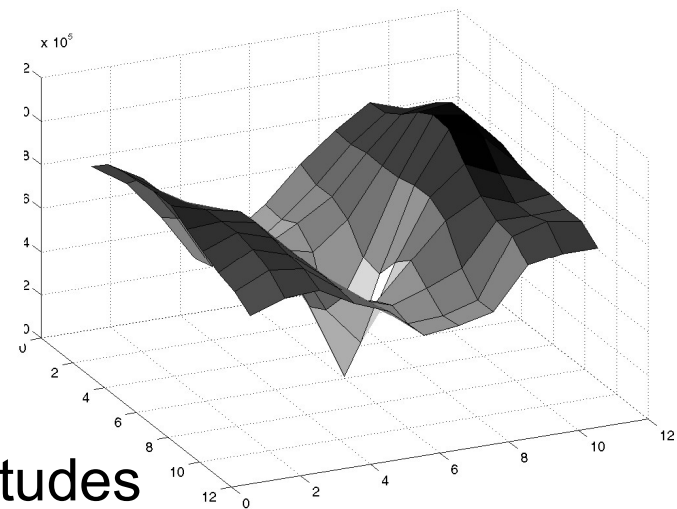


High textured region



$$\sum \nabla I (\nabla I)^T$$

- gradients are different, large magnitudes
- large λ_1 , large λ_2



Challenges in Lukas-Kanade

- What are the potential causes of errors in this procedure?
 - Suppose $A^T A$ is easily invertible
 - Suppose there is not much noise in the image

When our assumptions are violated

- Brightness constancy is not satisfied
- The motion is not small
- A point does not move like its neighbors
 - window size is too large
 - what is the ideal window size?

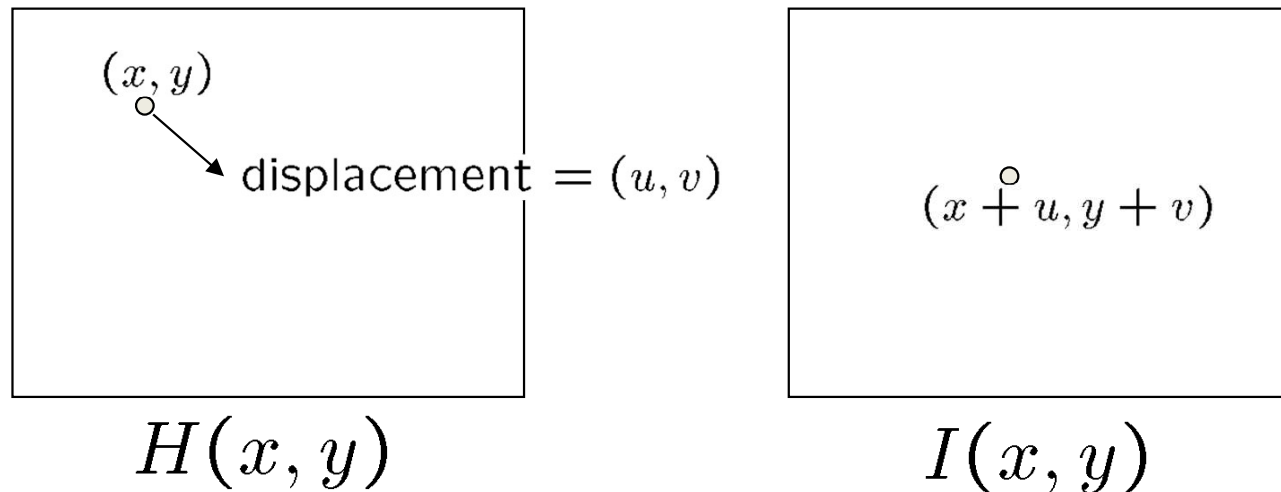
Outline

- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Iterative Lukas-Kanade Algorithm
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

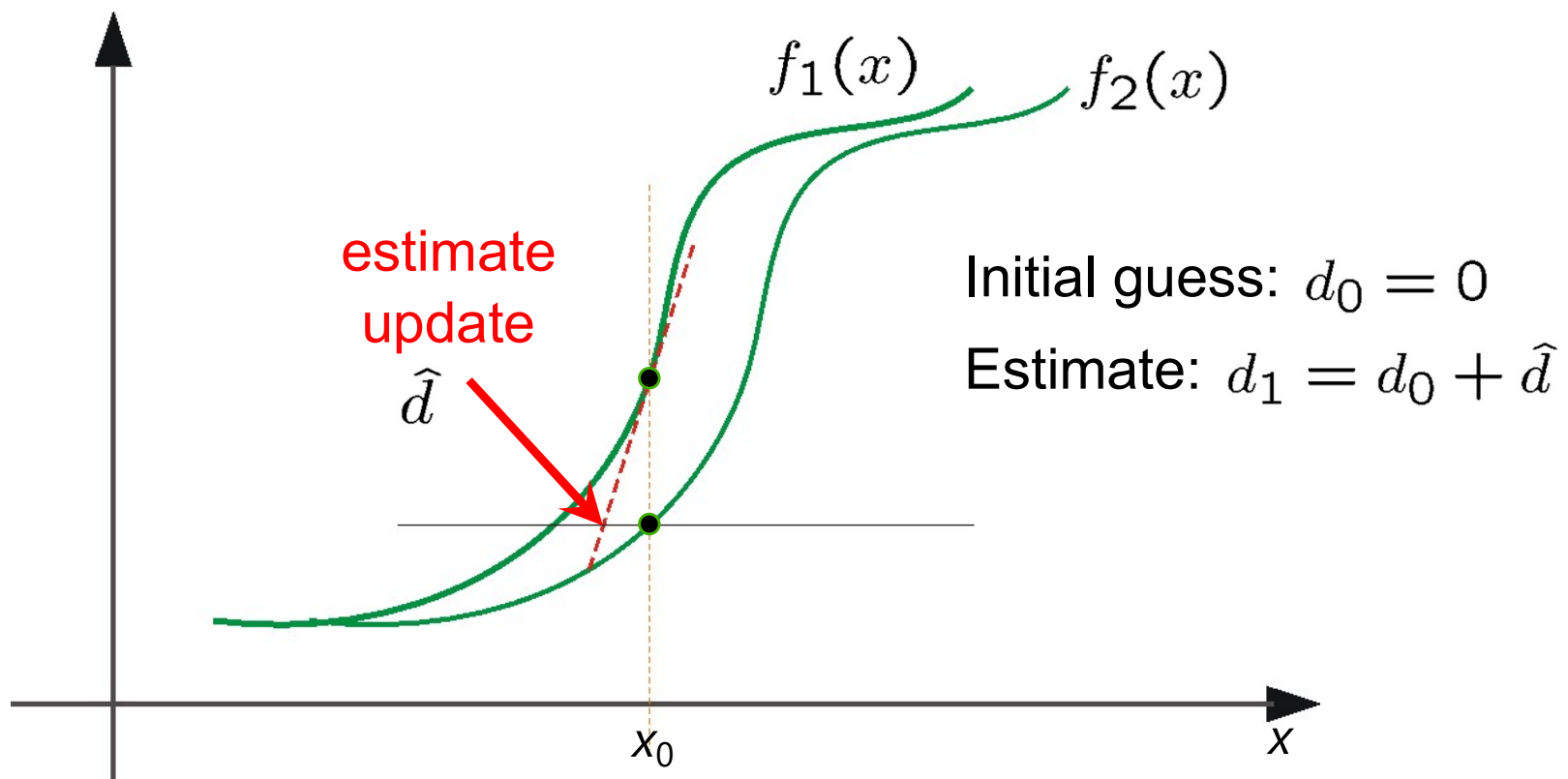
Iterative Refinement

Iterative Lukas-Kanade Algorithm

1. Estimate velocity at each pixel by solving Lucas-Kanade equations
2. Warp H towards I using the estimated flow field
 - *use image warping techniques (here, move every pixel to the new position according to the estimated optical flow)*
3. Repeat until convergence

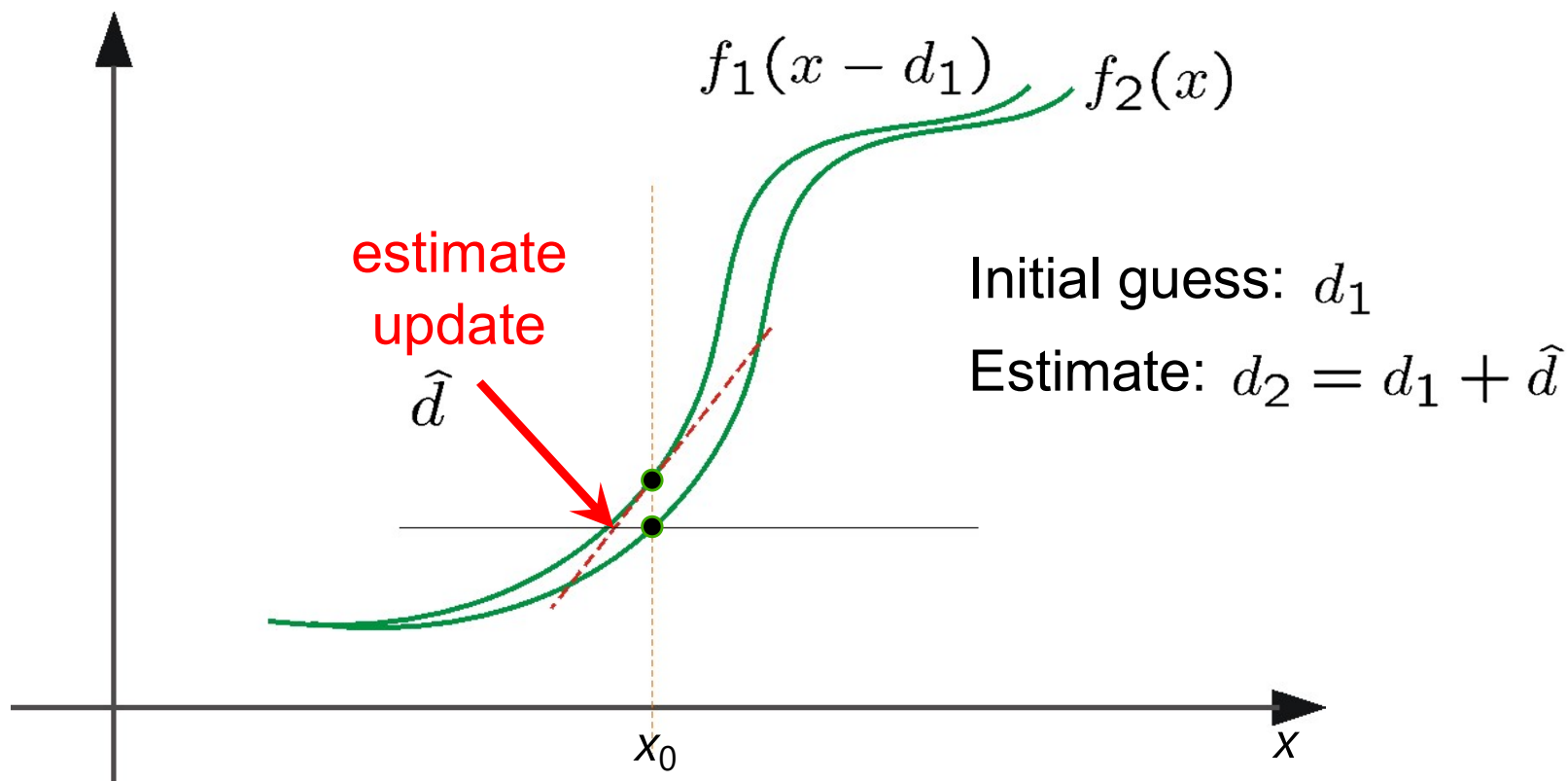


Optical Flow: Iterative Estimation

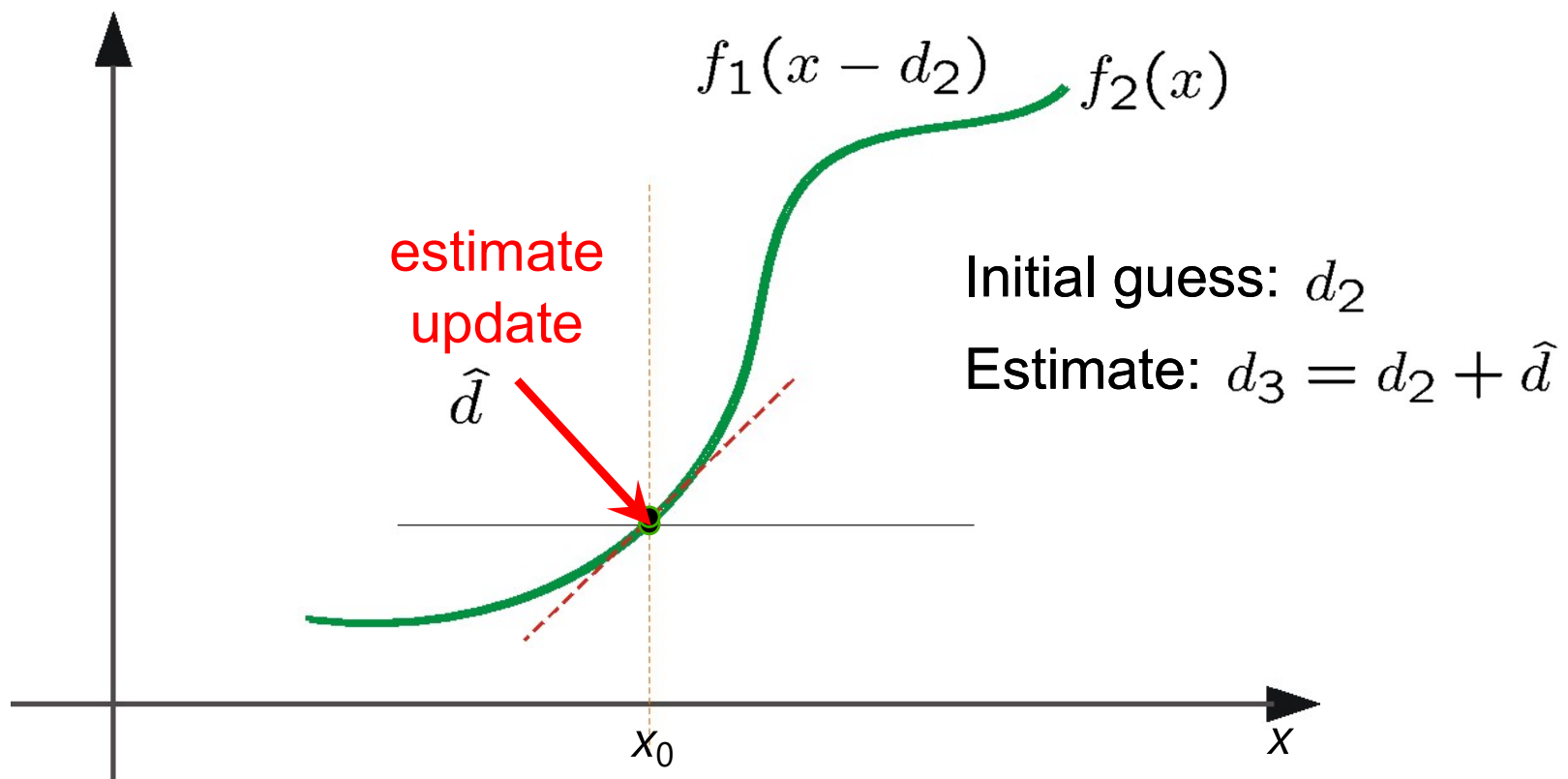


(using d for *displacement* here instead of u)

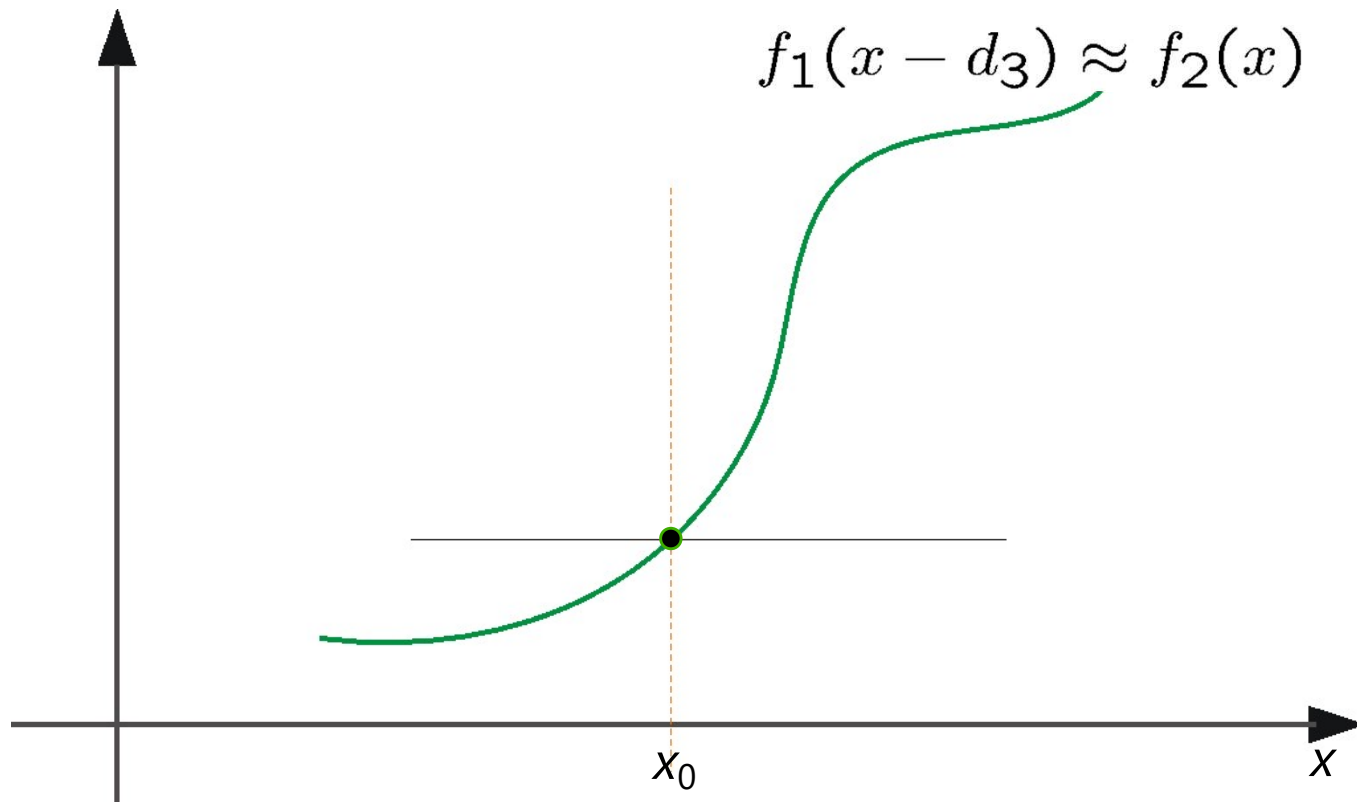
Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation



Revisiting the small motion assumption

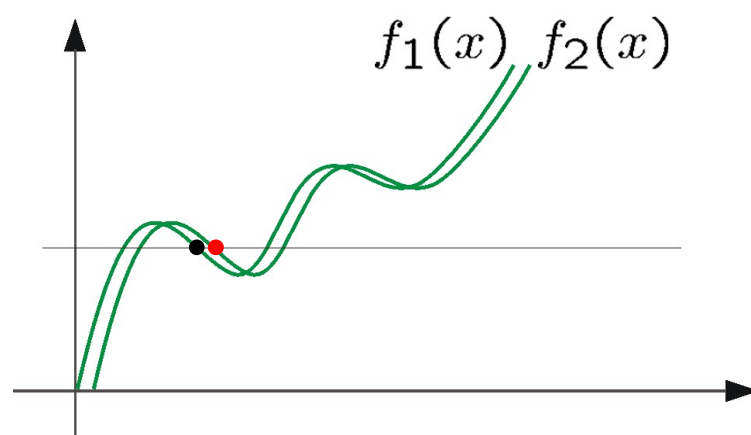


- Is this motion small enough?
 - Probably not—it's much larger than one pixel (2nd order terms dominate)
 - How might we solve this problem?

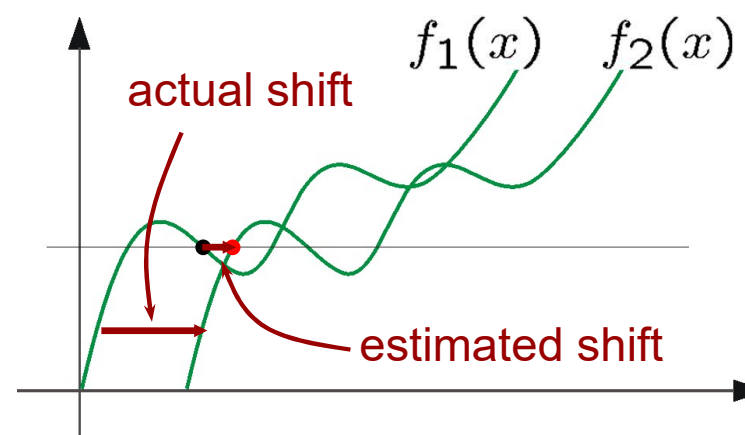
Optical Flow: Aliasing

Temporal aliasing causes ambiguities in optical flow because images can have many pixels with the same intensity.

I.e., how do we know which 'correspondence' is correct?



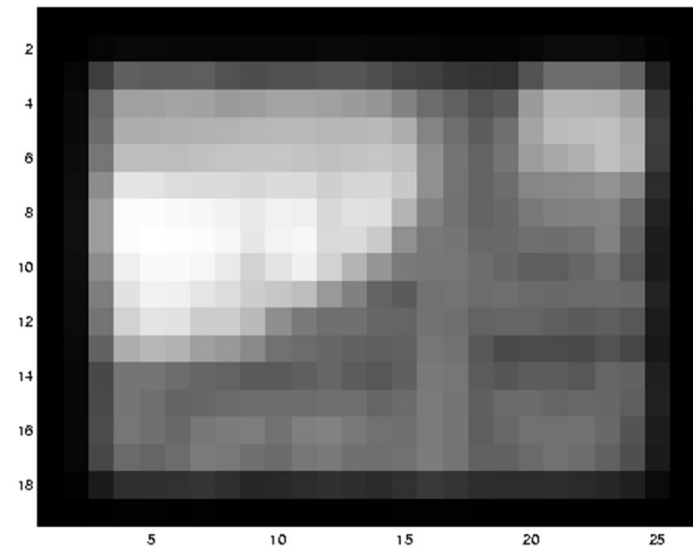
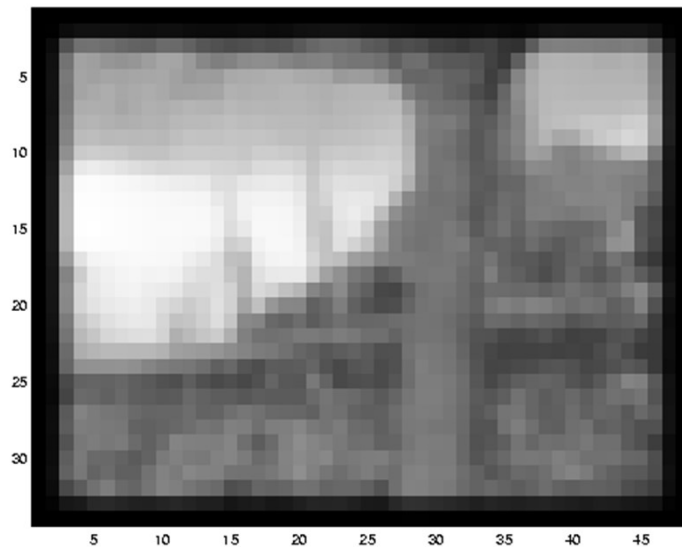
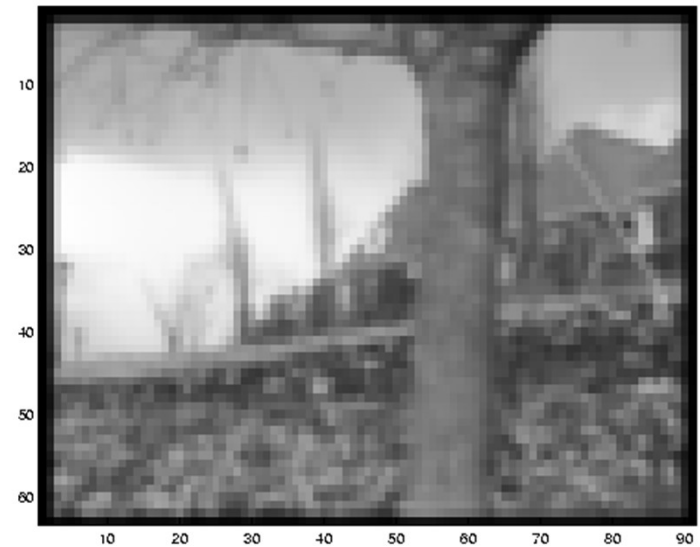
*nearest match is correct
(no aliasing)*



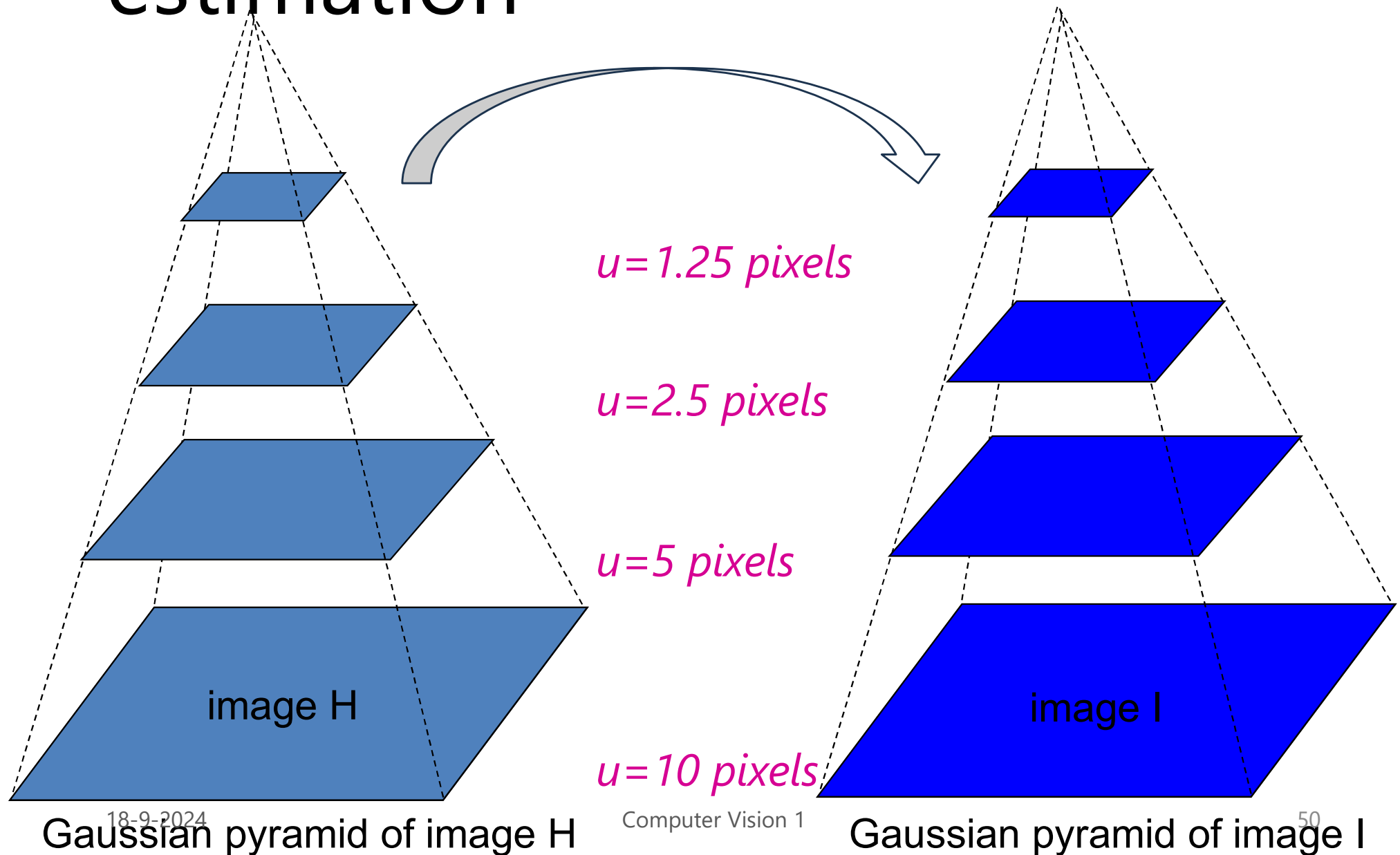
*nearest match is incorrect
(aliasing)*

To overcome aliasing: coarse-to-fine estimation.

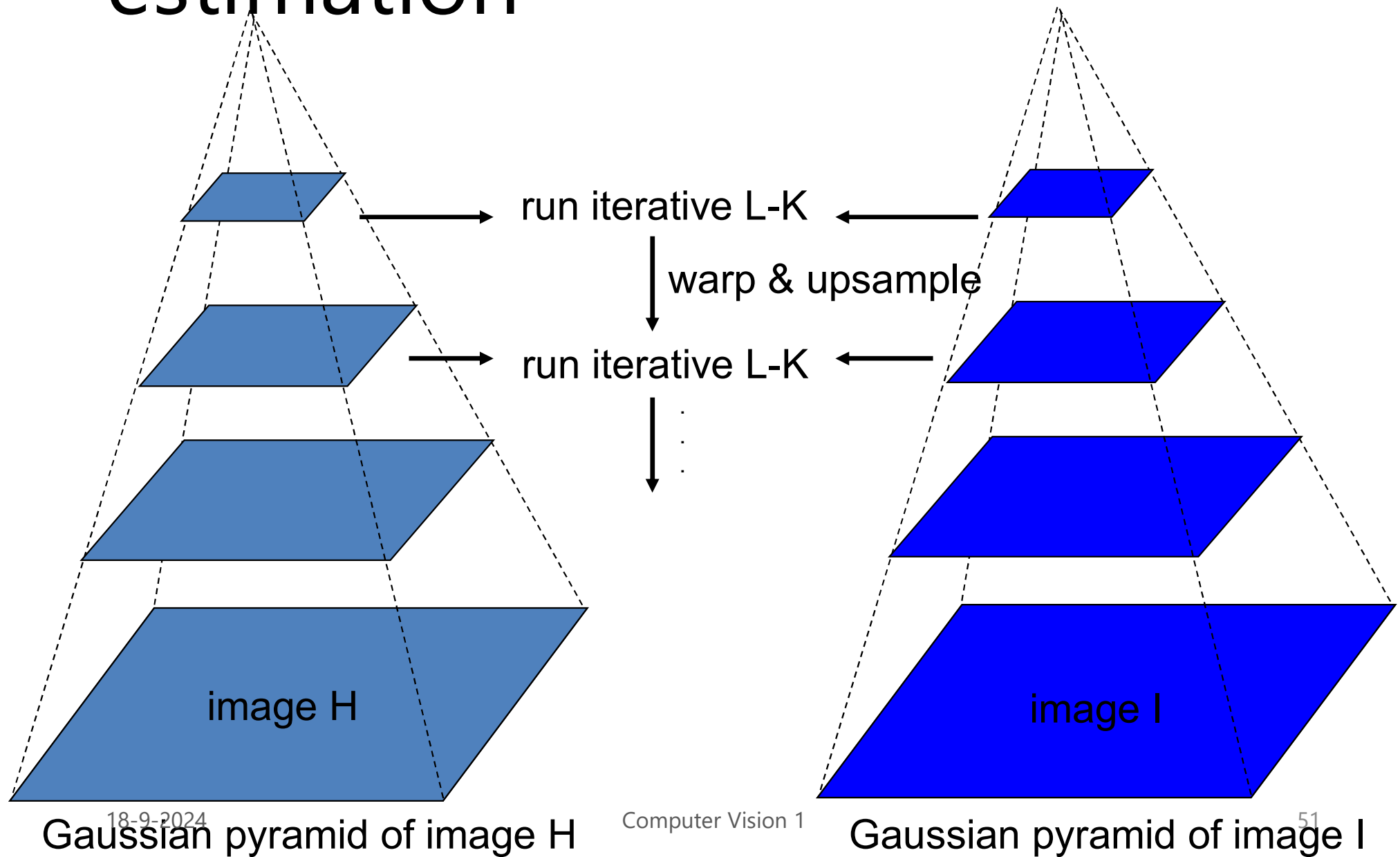
Reduce the resolution!



Coarse-to-fine optical flow estimation

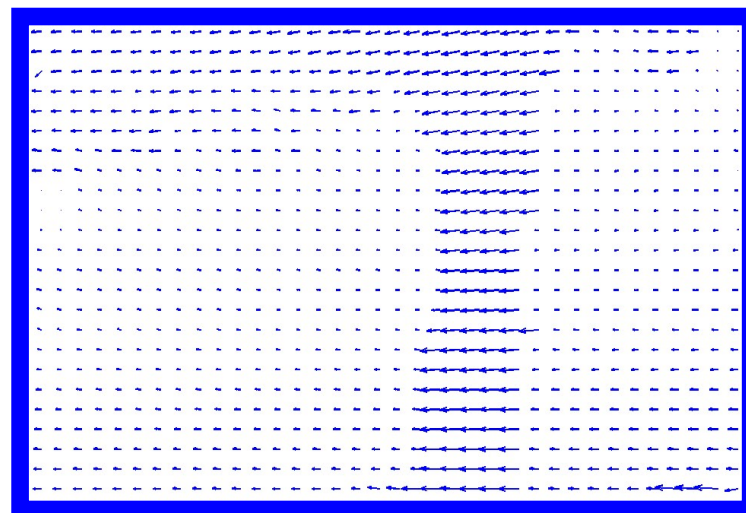


Coarse-to-fine optical flow estimation

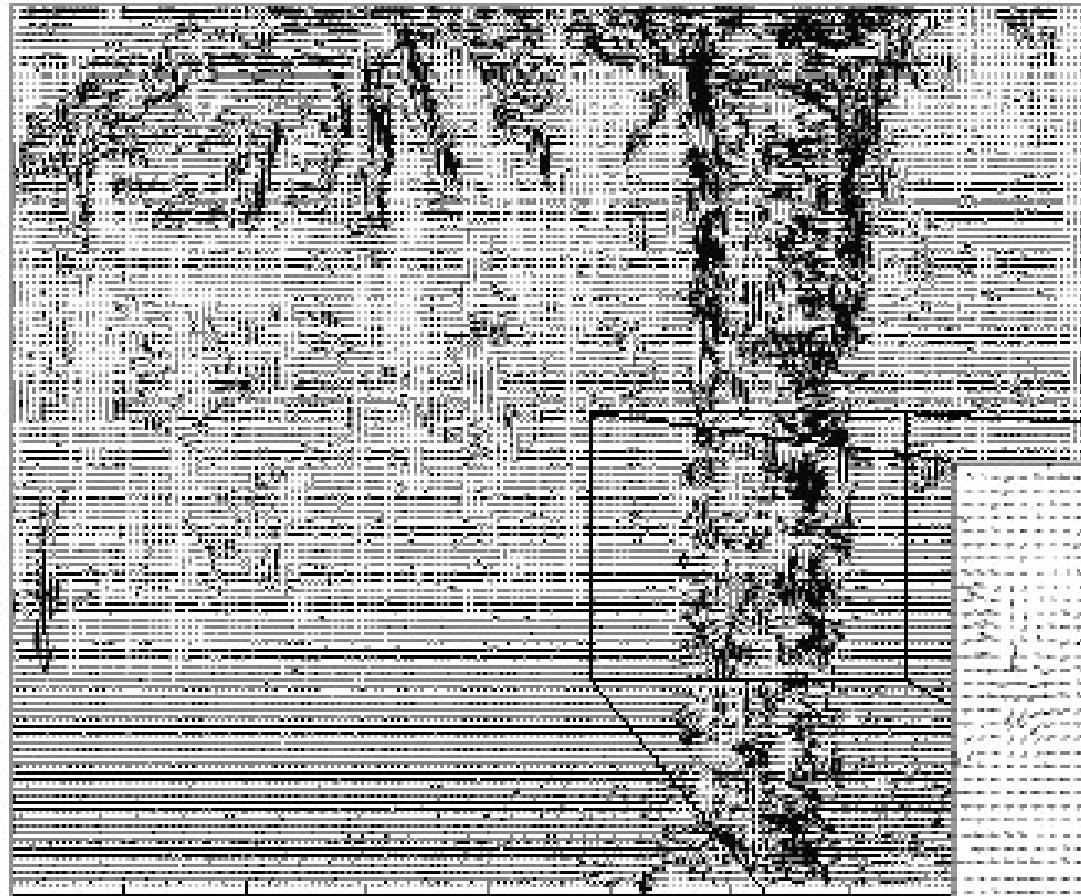


The Flower Garden Video

What should the
optical flow be?

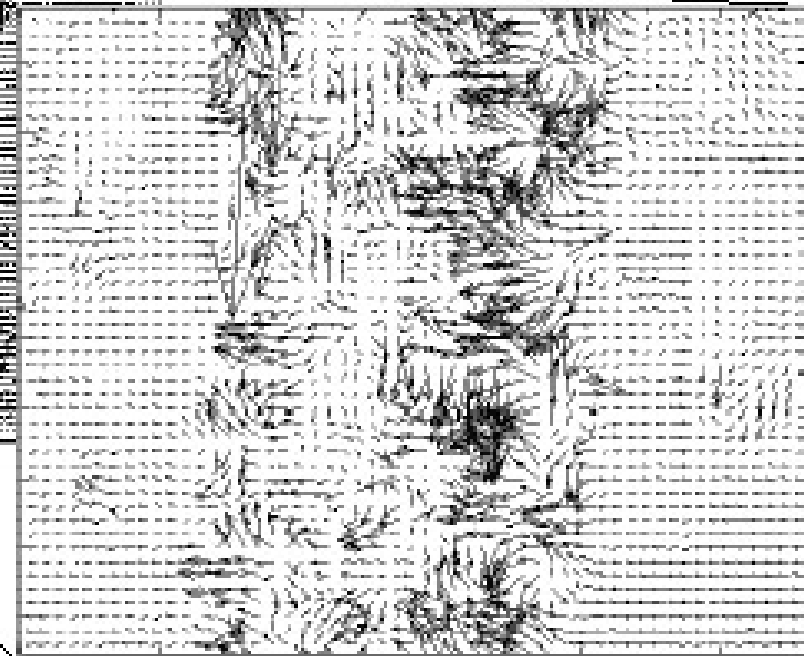


Optical Flow Results

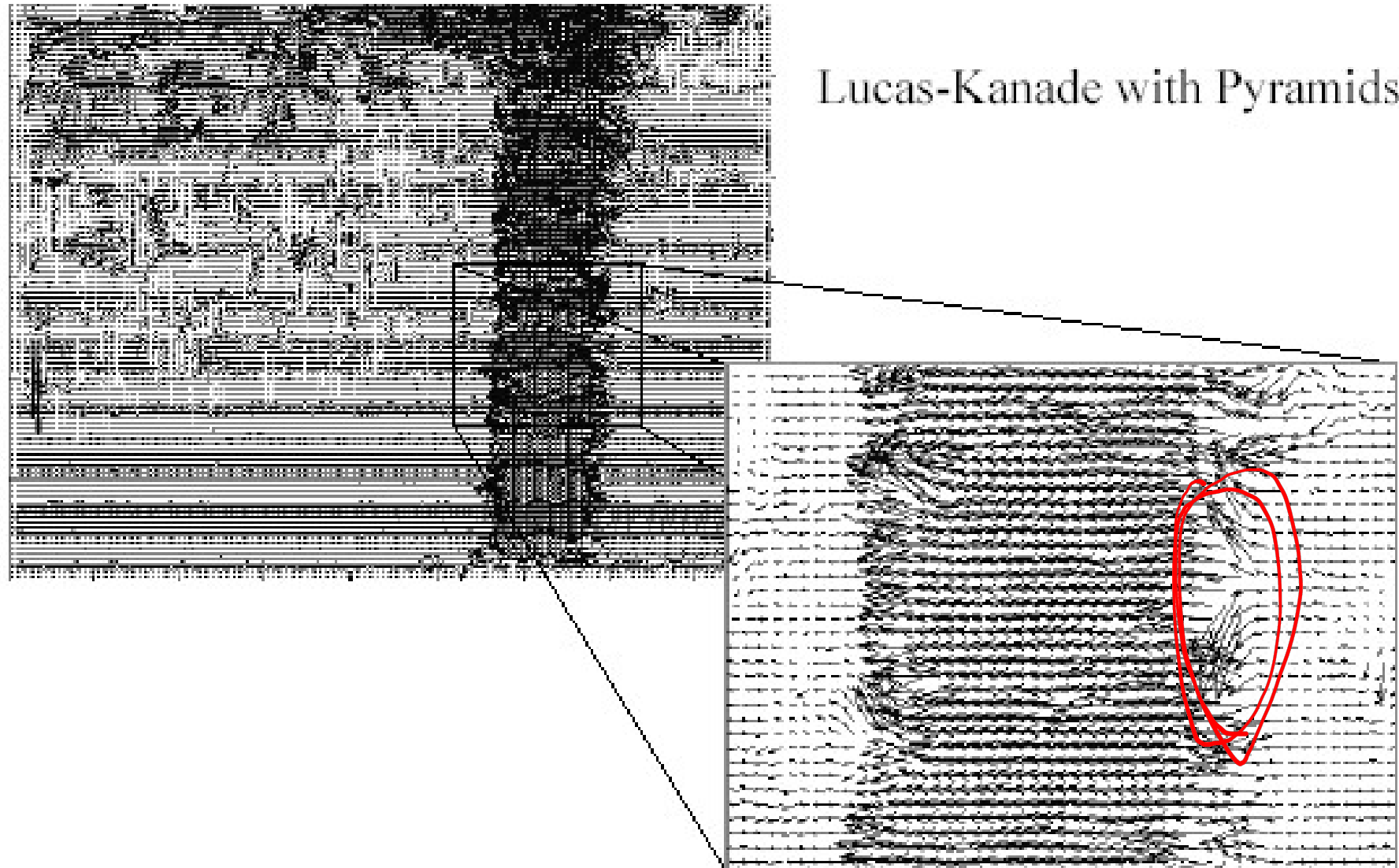


Lucas-Kanade
without pyramids

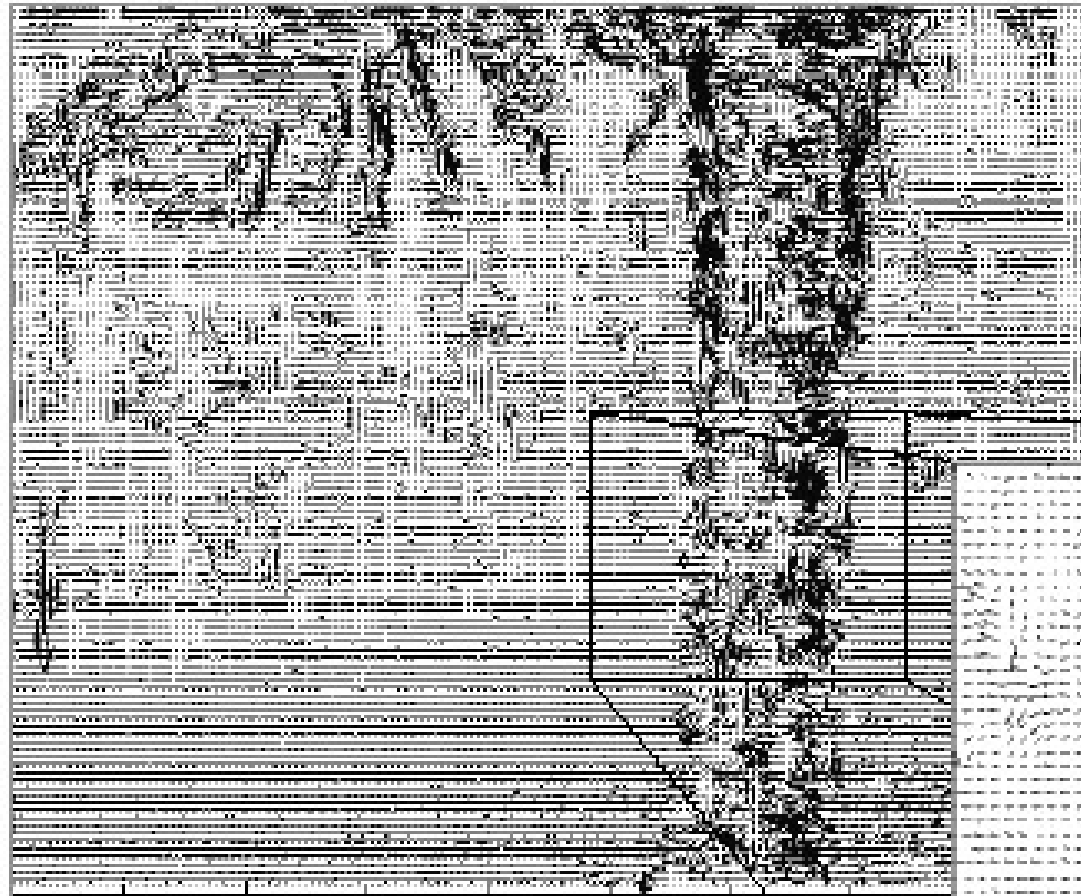
Fails in areas of large
motion



Optical Flow Results

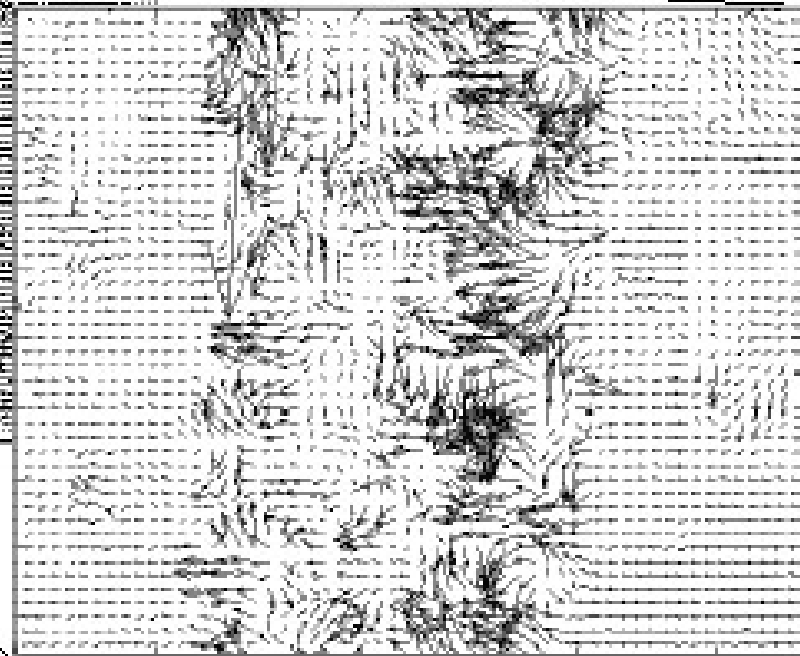


Optical Flow Results

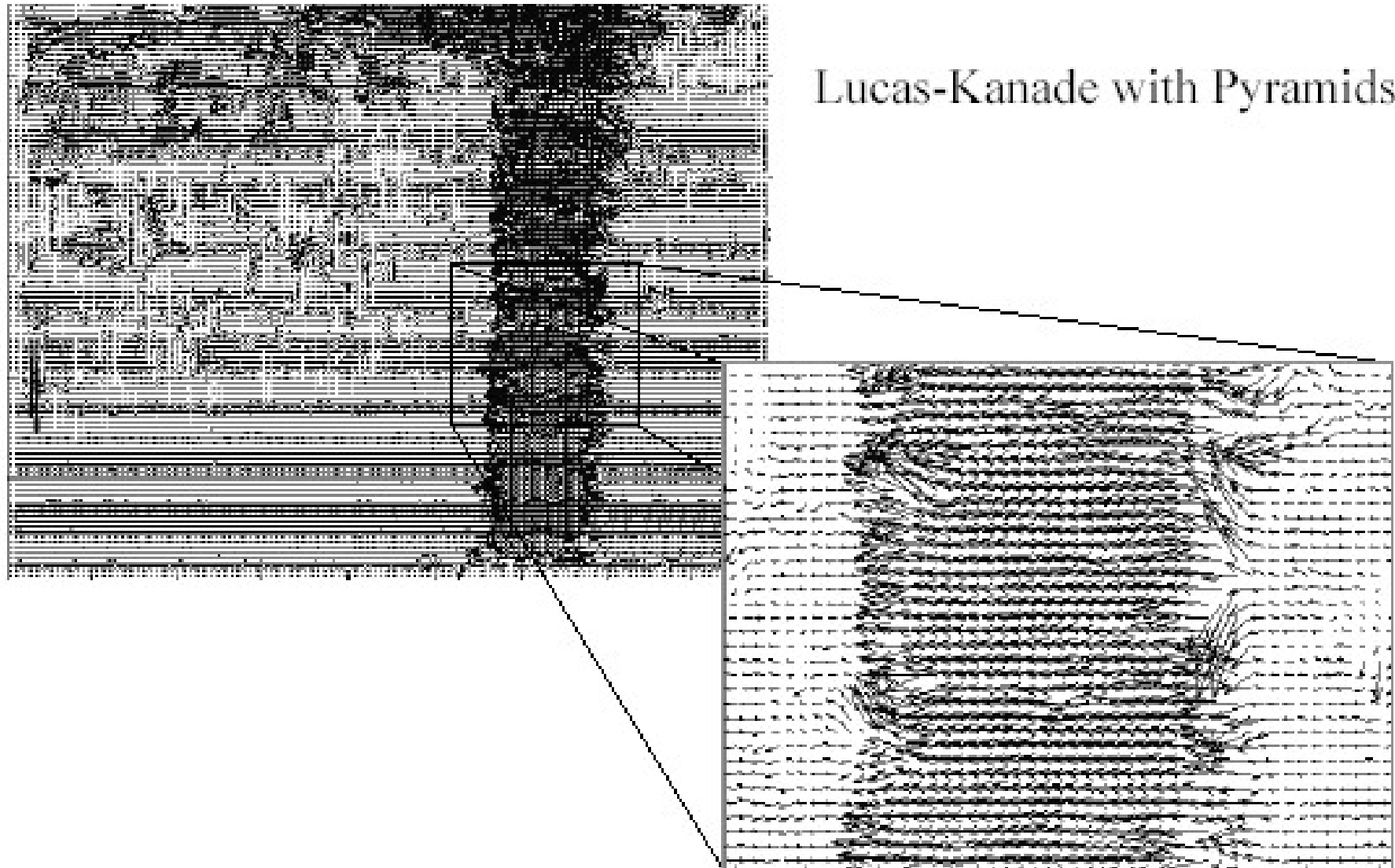


Lucas-Kanade
without pyramids

Fails in areas of large
motion



Optical Flow Results



Lucas-Kanade flow in color

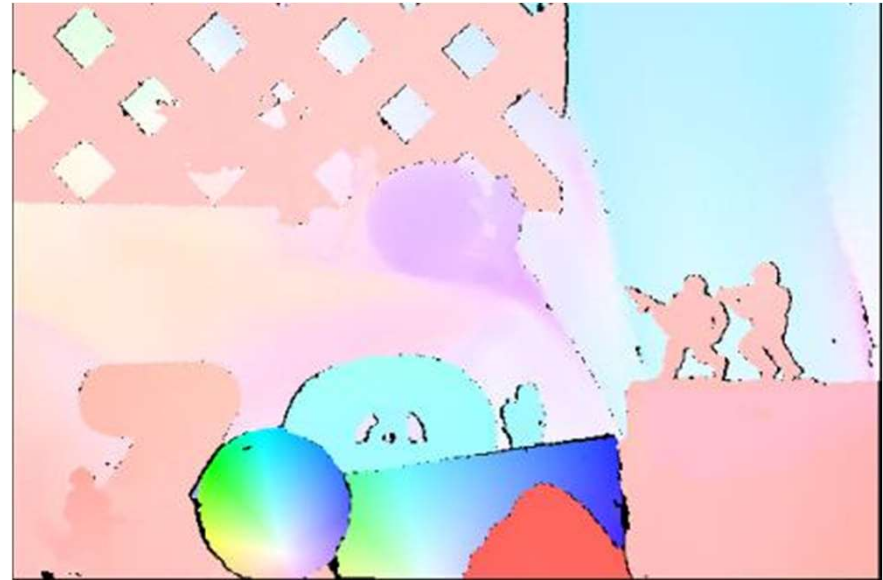


Lucas-Kanade flow in color

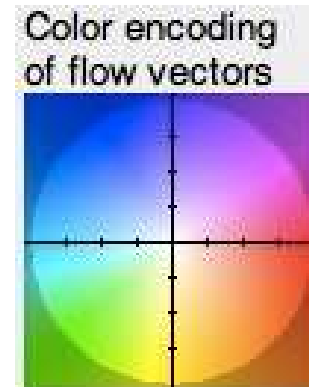


Flow quality evaluation

- Middlebury dataset
 - <http://vision.middlebury.edu/flow/>

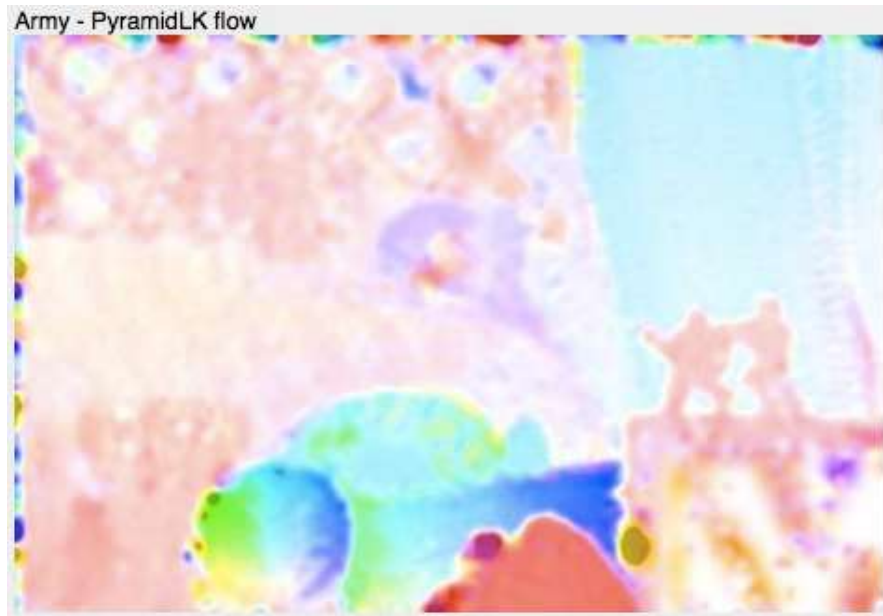


Ground Truth

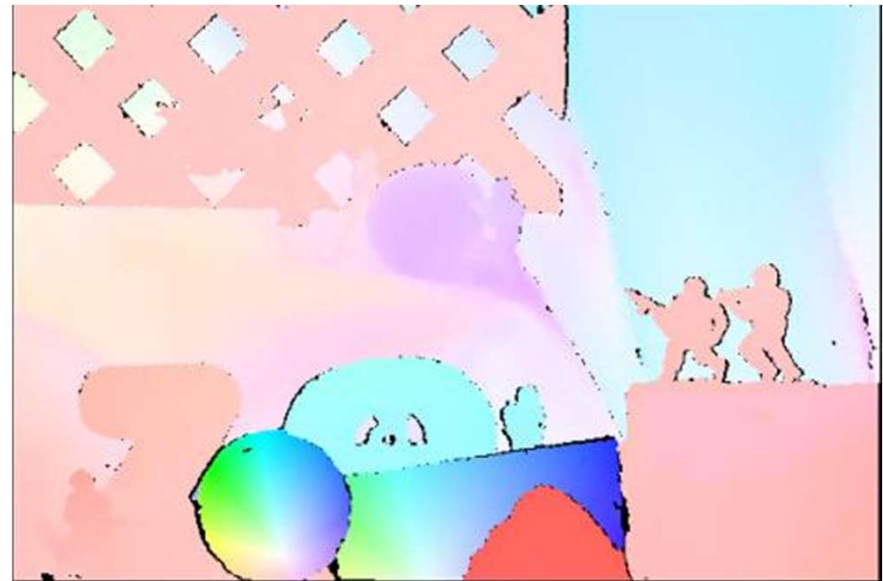


Flow quality evaluation

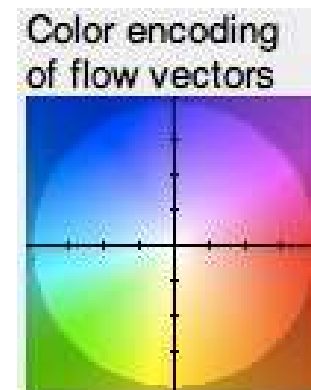
- Middlebury flow page
 - <http://vision.middlebury.edu/flow/>



Lucas-Kanade flow



Ground Truth

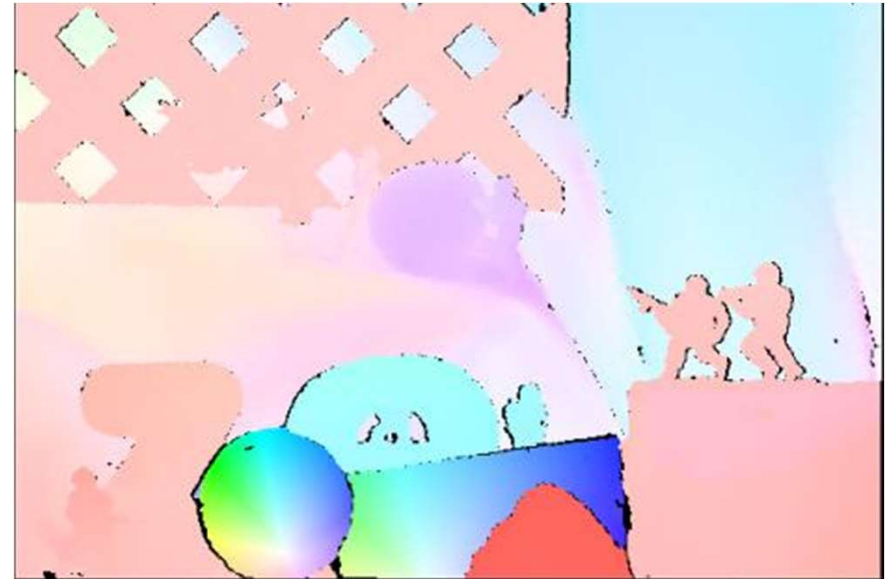


Flow quality evaluation

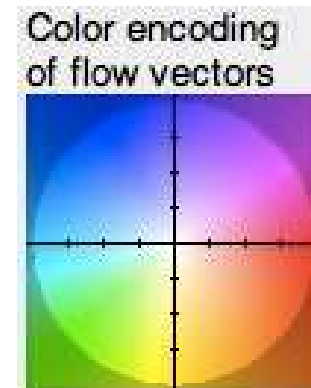
- Middlebury flow page
 - <http://vision.middlebury.edu/flow/>



Best-in-class alg



Ground Truth





18-9-2024

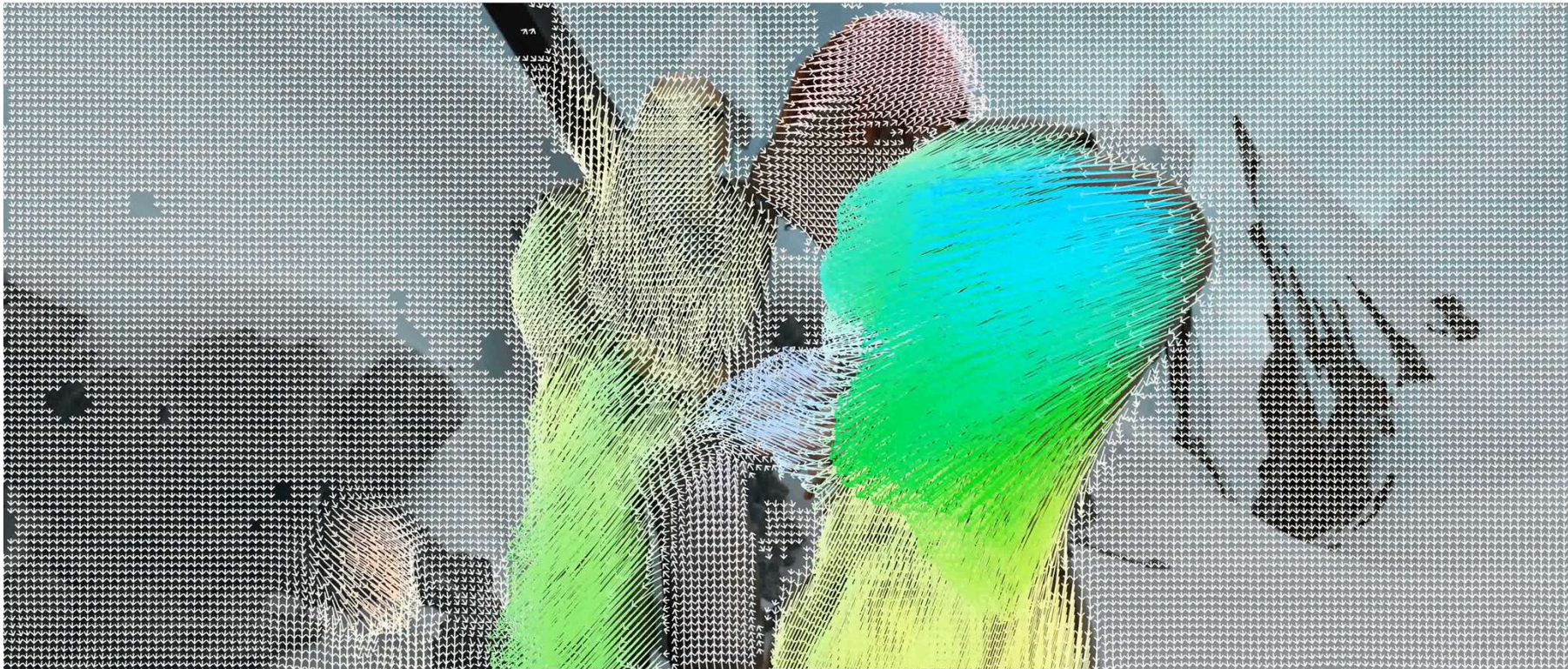
Computer Vision 1

Video stabilization



Volumetric Correspondence Networks for Optical Flow

NeurIPS 2019 Gengshan Yang & Deva Ramana
Robotics Institute, Carnegie Mellon University



Outline

- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

Previously: Features!

- Highly descriptive local regions
- Ways to describe those regions
- Useful for:
 - Matching
 - Recognition
 - Detection

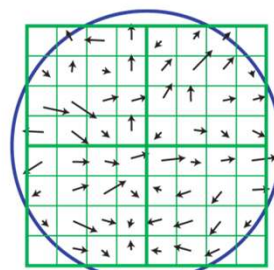
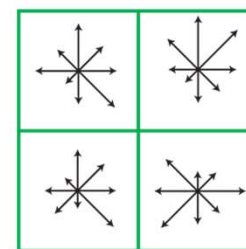
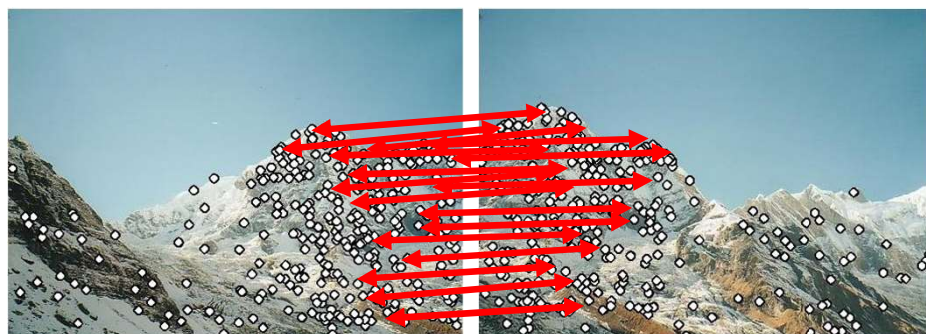
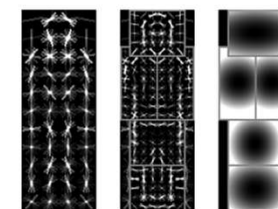
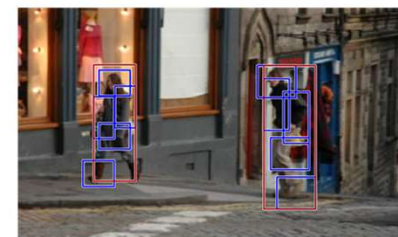


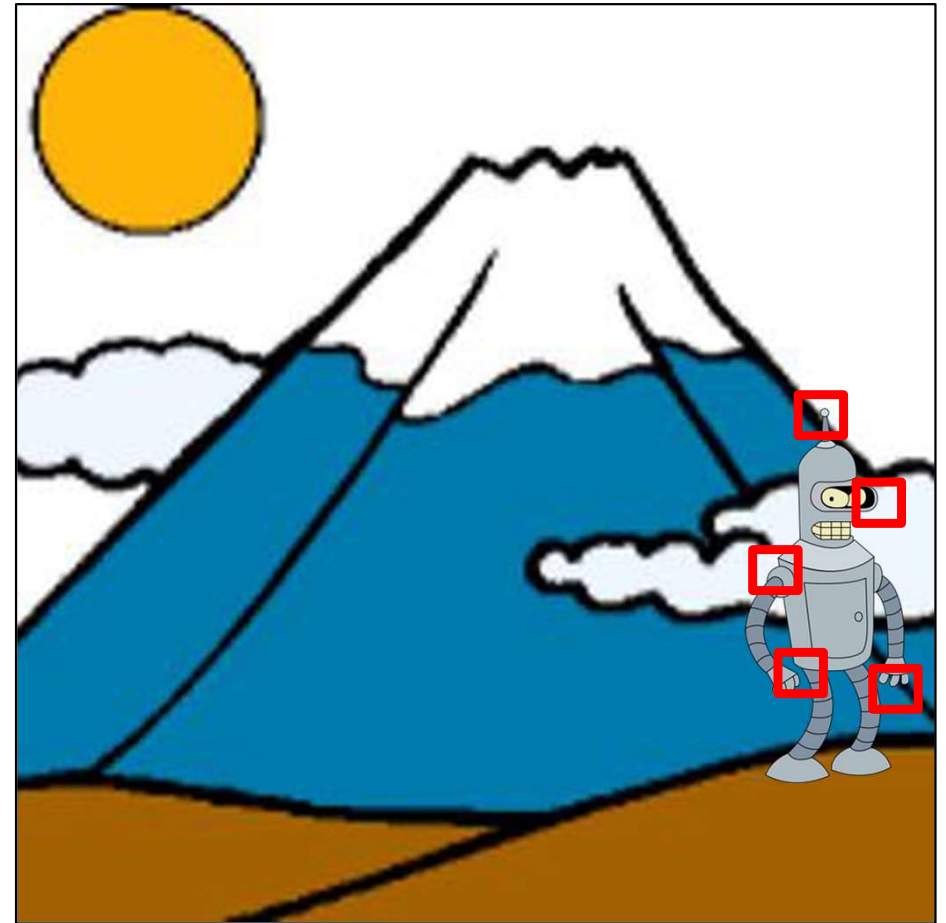
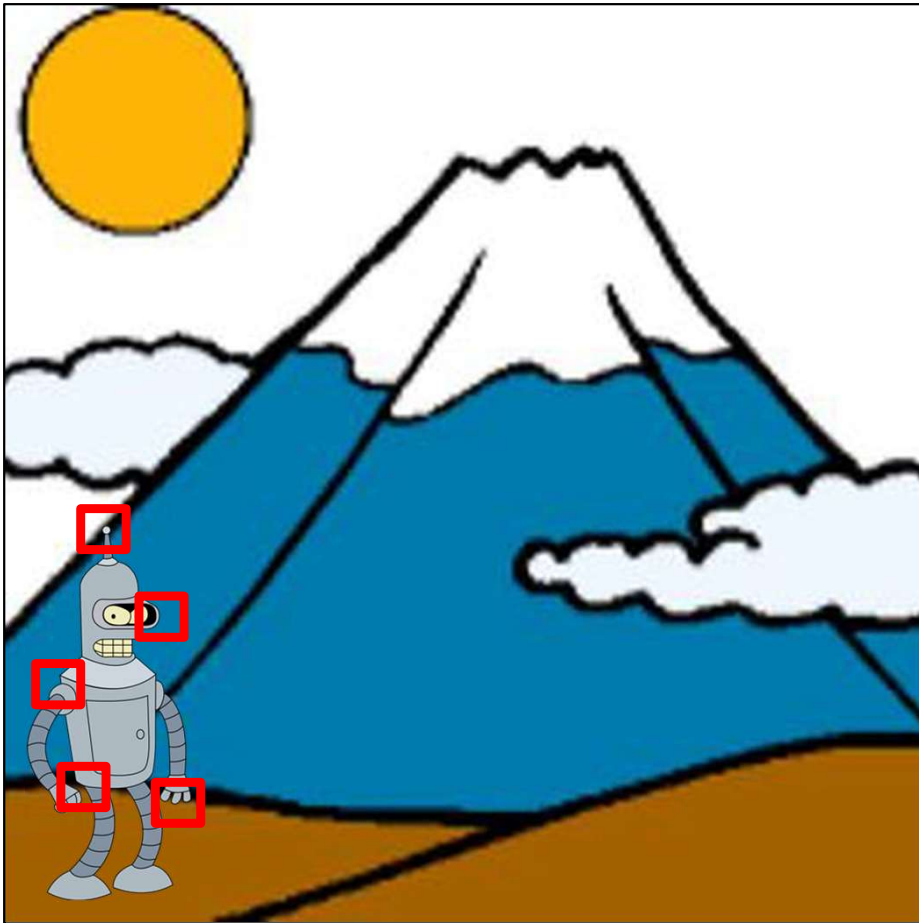
Image gradients



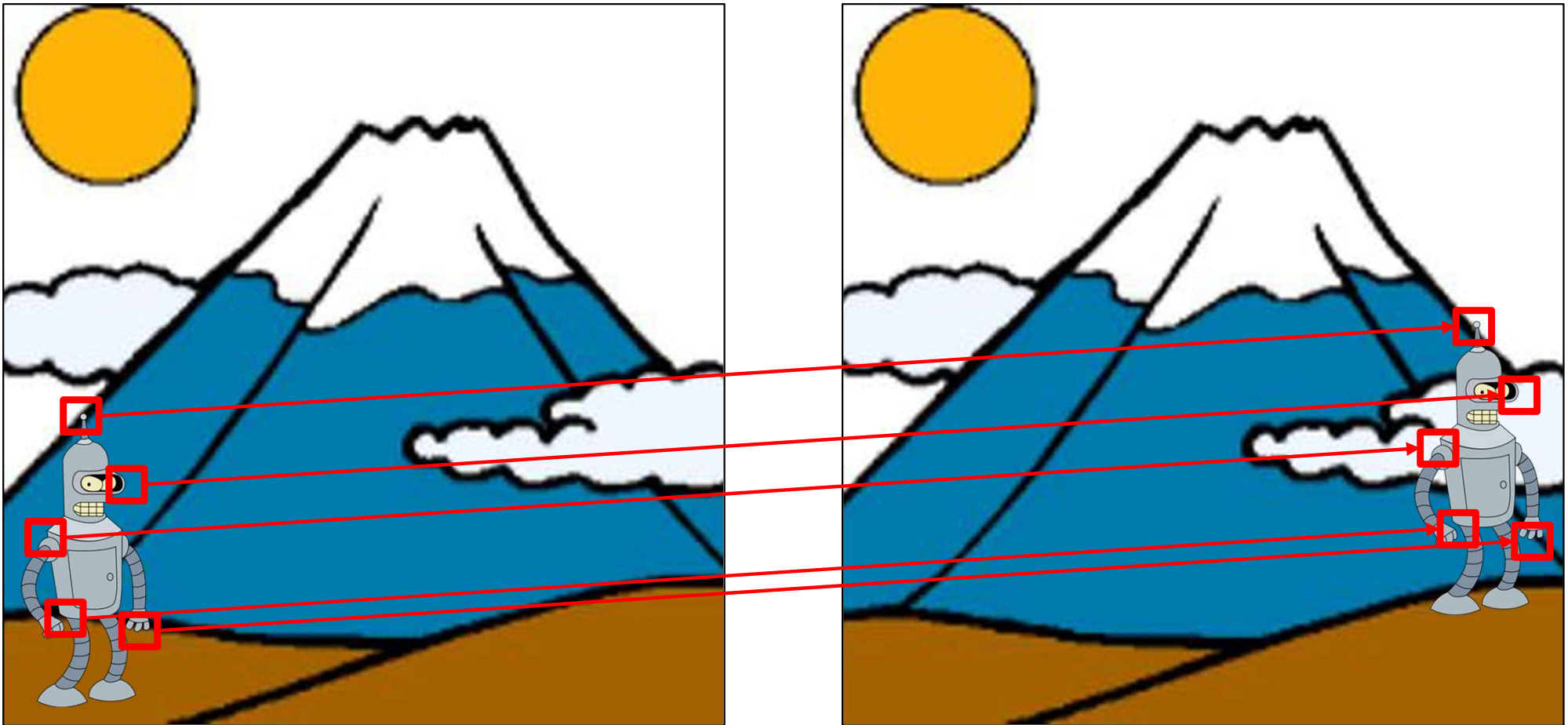
Keypoint descriptor



Feature Matching



Feature Matching

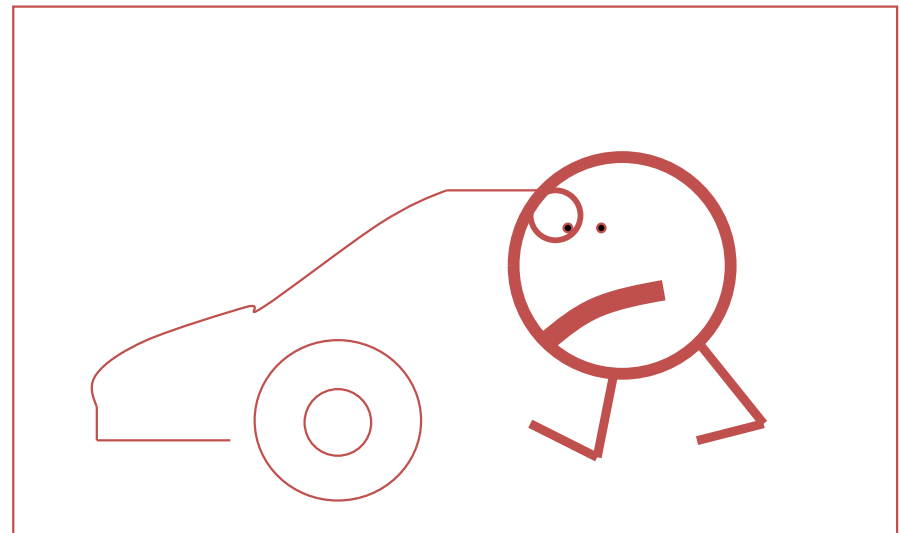


Template based Tracking

- Tracking consists in searching for the target object in a frame by comparing with a **template** image.
- We assume that the template is fixed and given in advance.



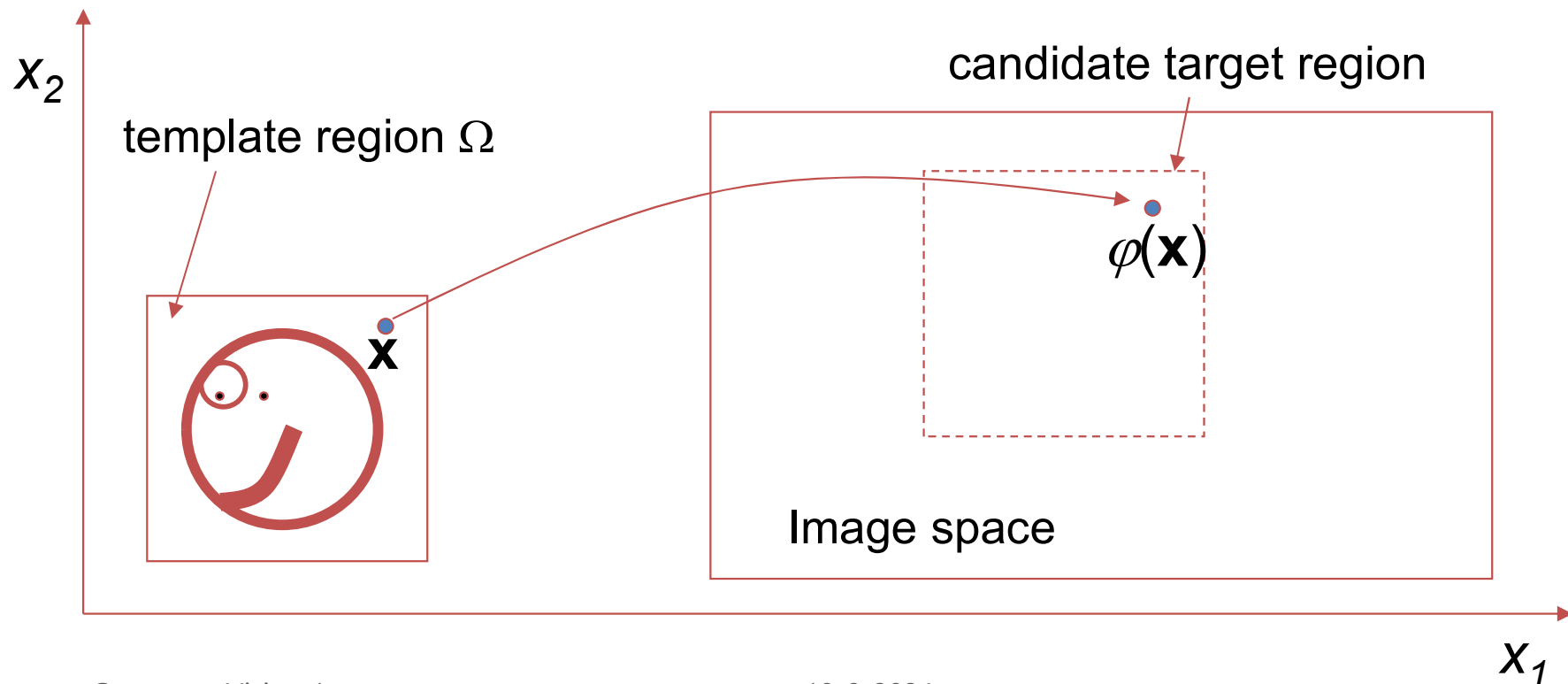
Template image
 $T(\mathbf{x})$



$I(\mathbf{x},t)$

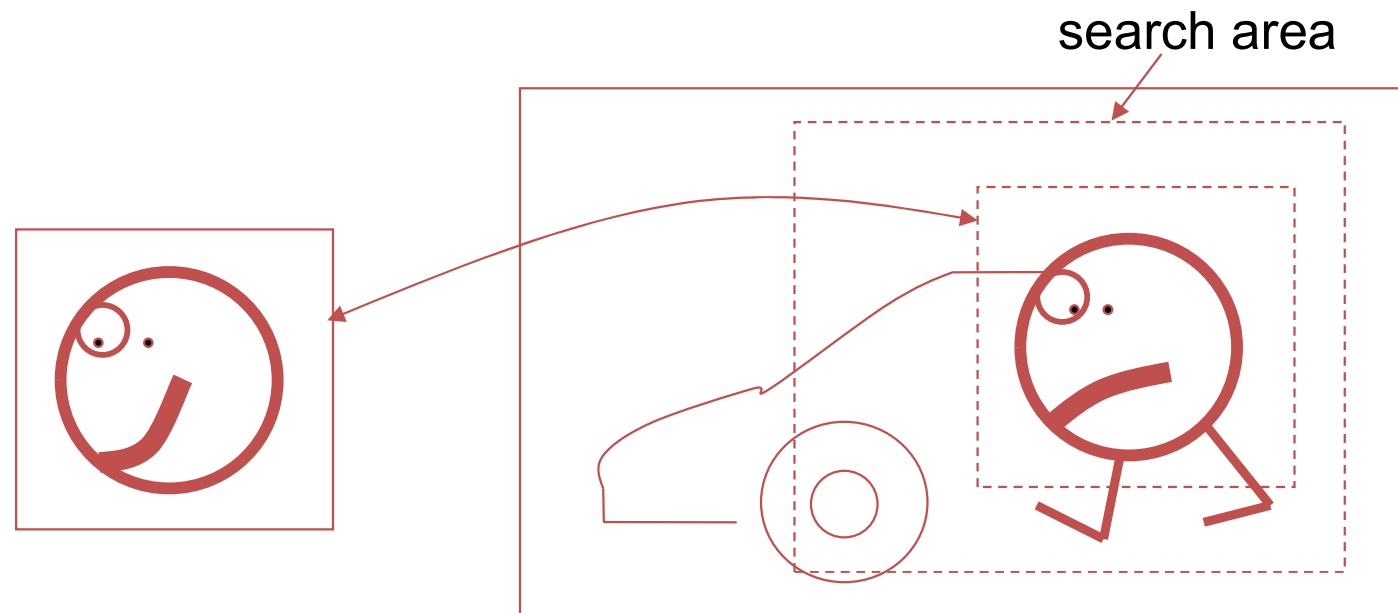
Template to Target Transformation

- The template is mapped into a candidate target region the image using a transformation of coordinates: $\phi(\mathbf{x})$. This transformation depends on a parameter vector $\mathbf{y}$. Different candidate regions correspond to different values of $\mathbf{y}$. So we write $\phi(\mathbf{x}; \mathbf{y})$.



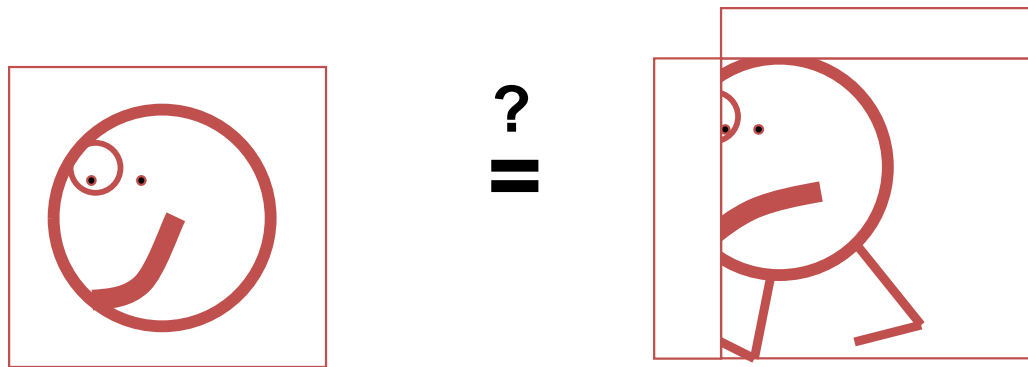
Search Space

- Align the template with every possible candidate region in the image, and find the most similar candidate according to a **similarity measure**.
- We search the target only in an area around the previous position exploiting general knowledge that the object won't have moved far.



Similarity Measure

- We need a measure of how similar (or far apart) the template and the candidate are.



- The similarity measure can be based on:
 - pixelwise intensity (color) difference: **SSD** and **correlation** trackers,
 - histogram difference: **mean-shift** tracker.

SSD and Correlation

- SSD is short for sum-of-squared-difference:

$$D(\mathbf{y}) = \sum_{\mathbf{x} \in \Omega} [I(\mathbf{x} + \mathbf{y}) - T(\mathbf{x})]^2 \rightarrow \min_{\mathbf{y}}$$

- A simpler similarity measure is the (unnormalized) cross-correlation:

$$C(\mathbf{y}) = \sum_{\mathbf{x} \in \Omega} I(\mathbf{x} + \mathbf{y})T(\mathbf{x}) \rightarrow \max_{\mathbf{y}}$$

Exhaustive Search

- Calculate SSD for every $\mathbf{y}$ in a search window and choose the position with the least SSD.
- Strengths: robustness and simplicity in implementation.
- Weaknesses:
 - Computations could be time-consuming in case of a large search window.
 - Only suitable for translation.

Gradient Descent

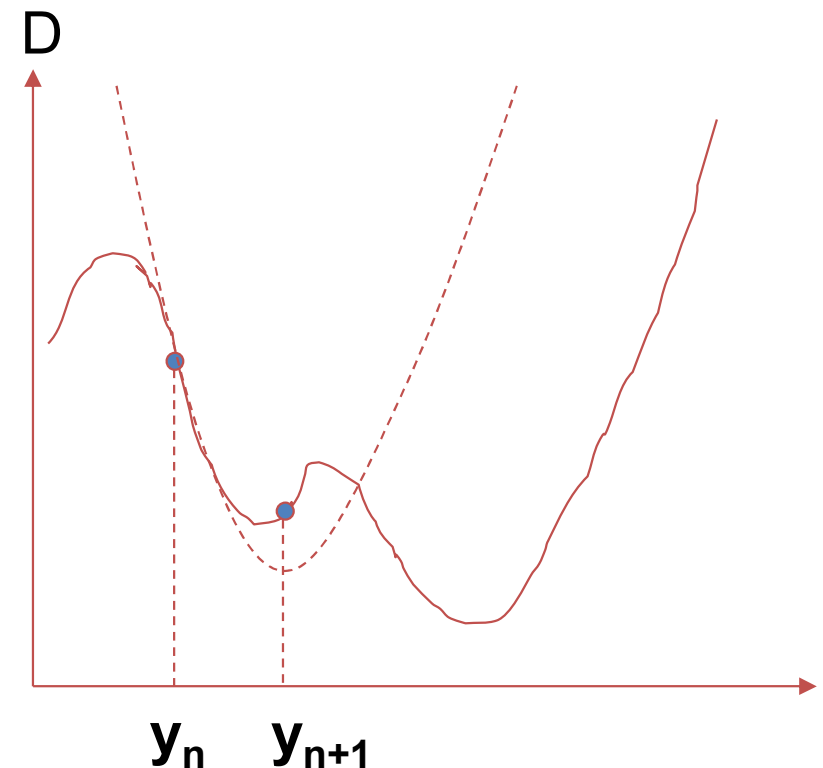
- Iteratively update the solution estimate using Newton's method:

$$D(\mathbf{y}_n + \Delta \mathbf{y}) \approx$$

$$D(\mathbf{y}_n) + \nabla D^T \Delta \mathbf{y} + \frac{1}{2} \Delta \mathbf{y}^T \nabla^2 D \Delta \mathbf{y} \rightarrow \min_{\Delta \mathbf{y}}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n - [\nabla^2 D]^{-1} \nabla D$$

- Efficient computation. Able to cope with rotation.
- Finds a local minimum only.





Outline

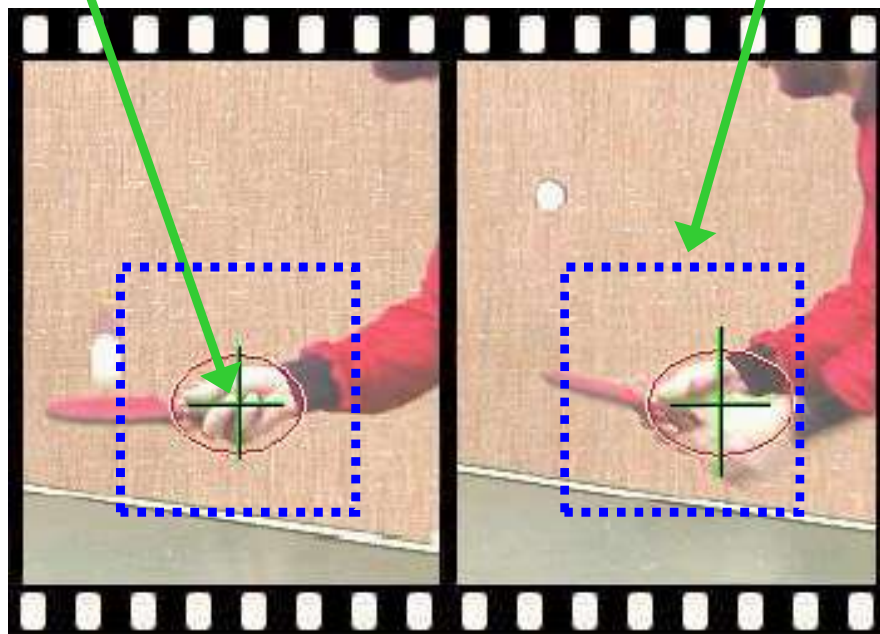
- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Iterative Lukas-Kanade Algorithm
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

Mean-Shift Object Tracking

Start from the position of the model in the current frame

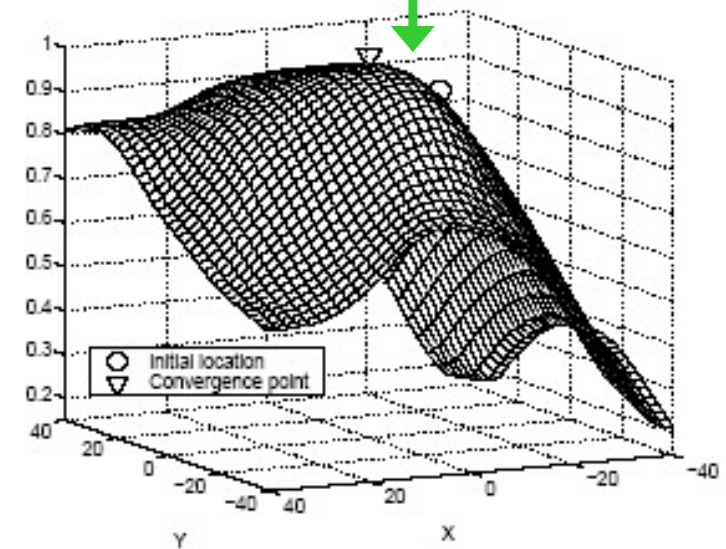
Search in the model's neighborhood in next frame

Find best candidate by maximizing a similarity func.



$\vec{q}$

$\vec{p}(y)$



$f[\vec{p}(y), \vec{q}]$

Mean-Shift Object Tracking

Target model: $\vec{q} = (q_1, \dots, q_m)$

Target candidate: $\vec{p}(y) = (p_1(y), \dots, p_m(y))$

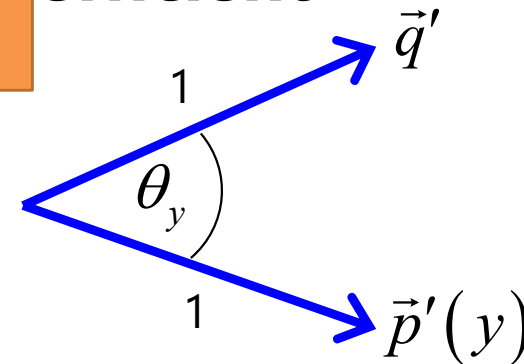
Similarity function: $f(y) = f[\vec{p}(y), \vec{q}] = ?$

The Bhattacharyya Coefficient is efficient

Beyond CV1

$$\vec{q}' = (\sqrt{q_1}, \dots, \sqrt{q_m})$$

$$\vec{p}'(y) = (\sqrt{p_1(y)}, \dots, \sqrt{p_m(y)})$$



$$f(y) = \cos \theta_y = \frac{\vec{p}'(y)^T \vec{q}'}{\|\vec{p}'(y)\| \cdot \|\vec{q}'\|} = \sum_{u=1}^m \sqrt{p_u(y) q_u}$$

Mean/shift Tracking Results



https://www.youtube.com/watch?v=_RviOAlqRyk

Meanshift tracking

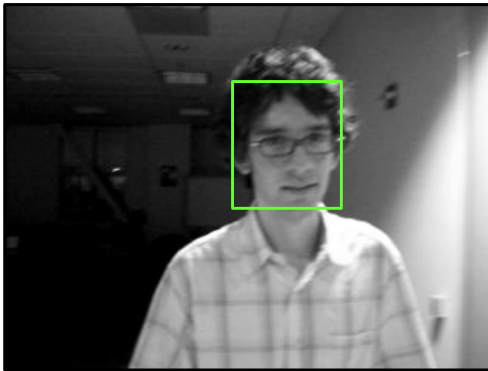


Outline

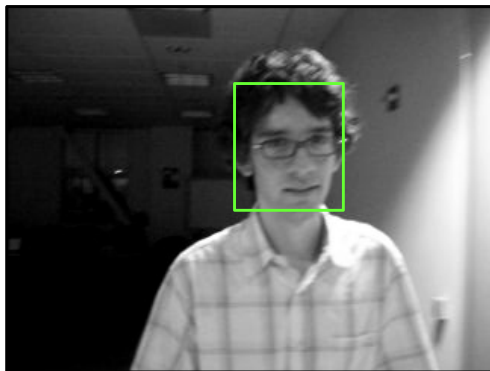
- Optical Flow and Motion
 - Motion
 - Optical Flow
 - Aperture Problem
 - Lukas-Kanade flow
 - Coarse-to-fine optical flow estimation
- Tracking
 - Template tracking
 - Mean-shift tracking
 - Tracking by Detection

Tracking by Detection

First frame is labeled



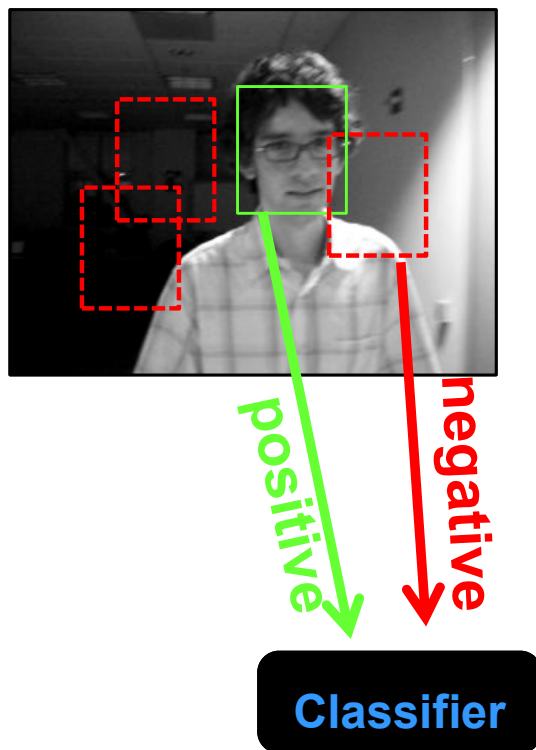
First frame is labeled



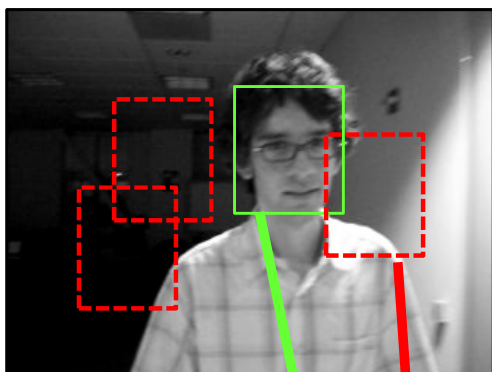
Classifier



Grab one **positive** patch, and some **negative** patch, and train/update the model.



Get next frame



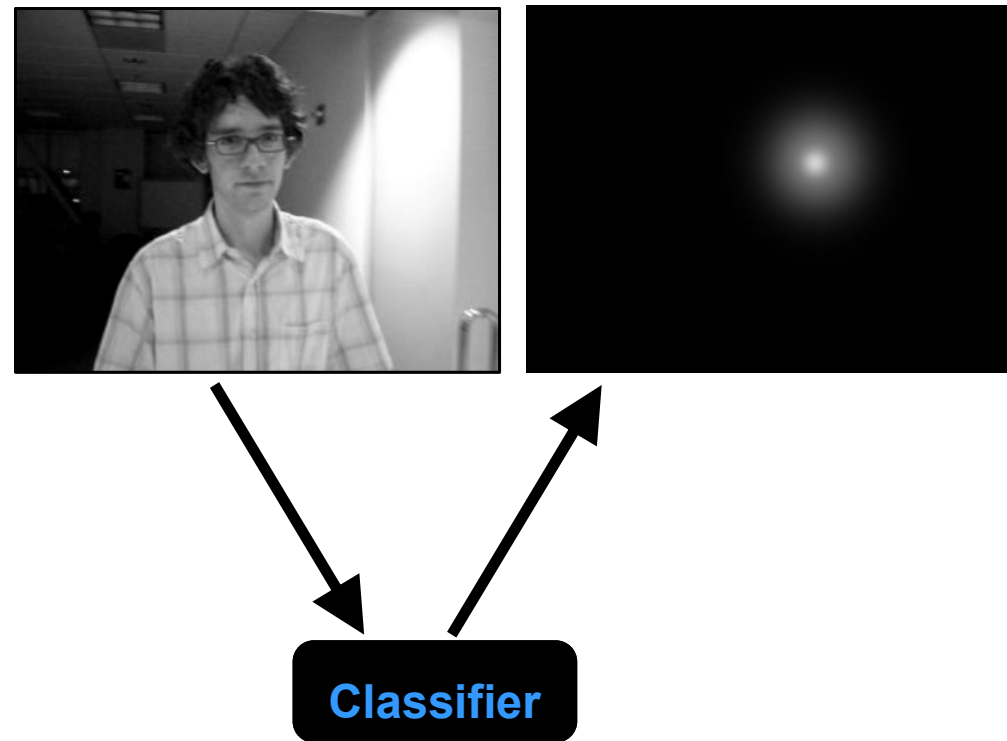
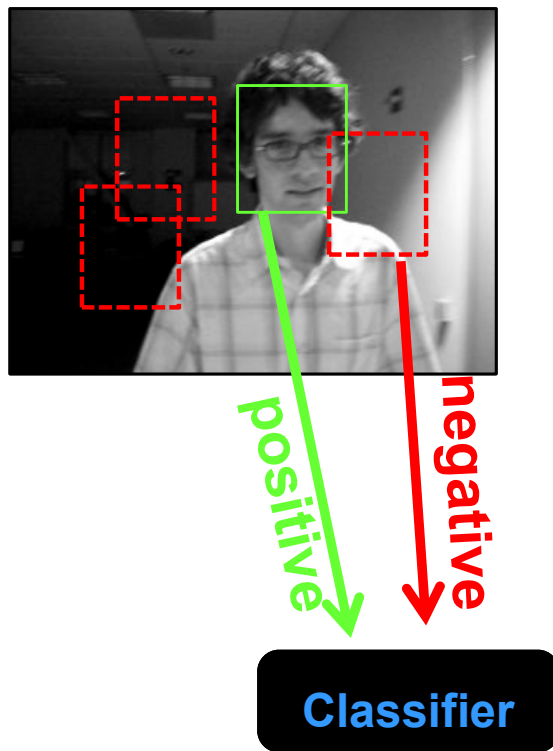
positive

negative

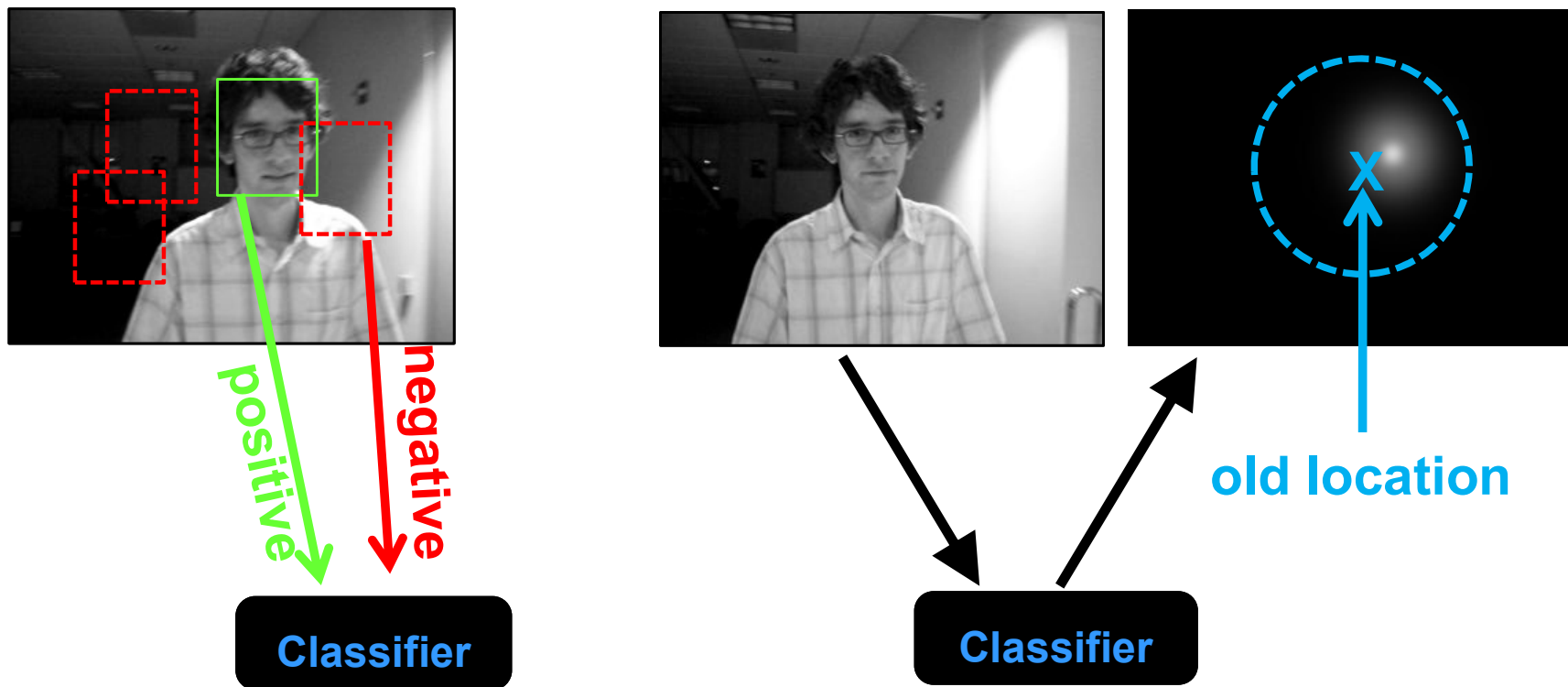
Classifier



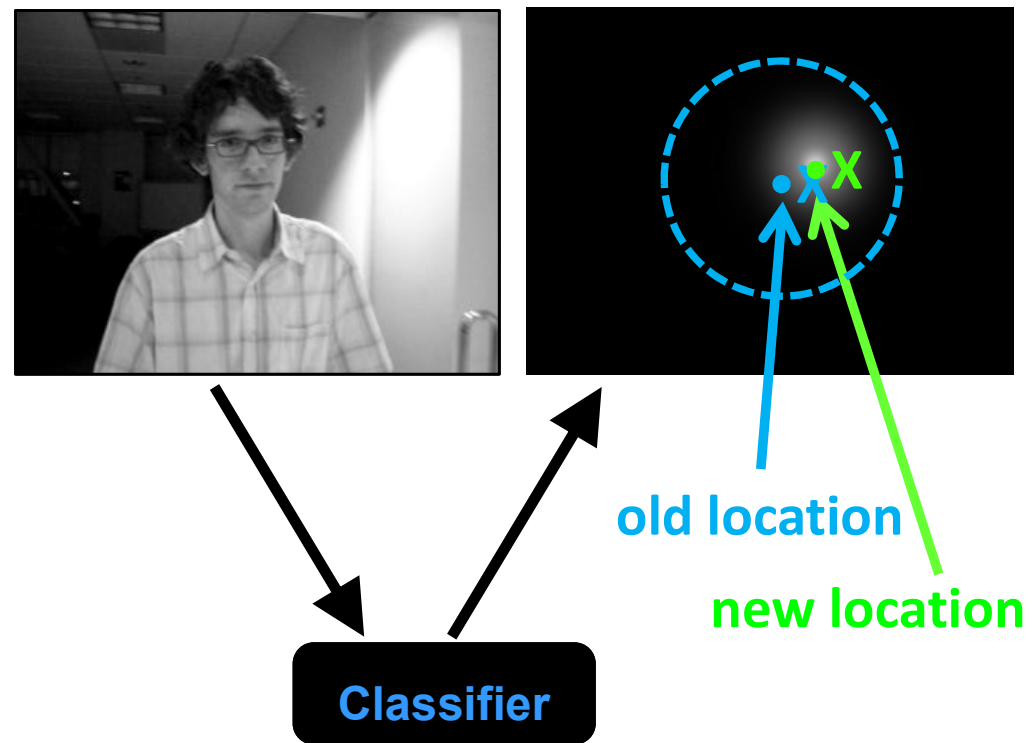
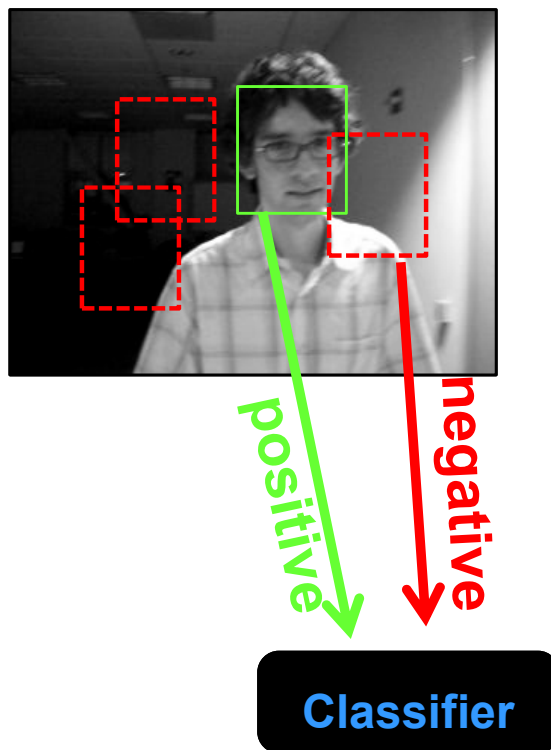
Evaluate classifier in some search window



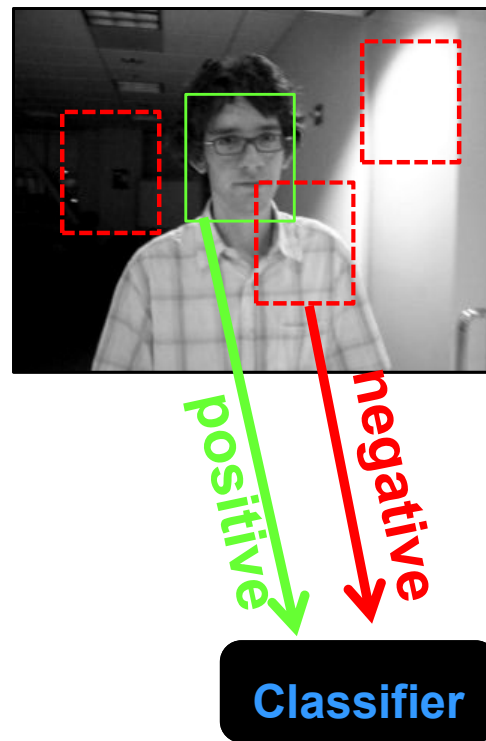
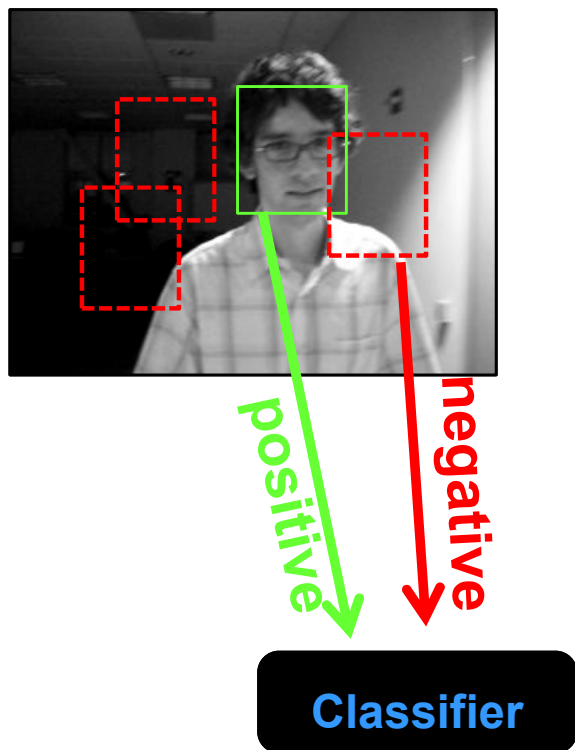
Evaluate classifier in some search window



Find max response



Repeat...



Tracking by Detection

Recent tracking work

- Focus on appearance model
- Borrow techniques from obj. detection
 - Slide a discriminative classifier around image
- **Adaptive** appearance model

Boris Babenko, Ming-Hsuan Yang, Serge Belongie, CVPR09

Recap: Classes of Techniques

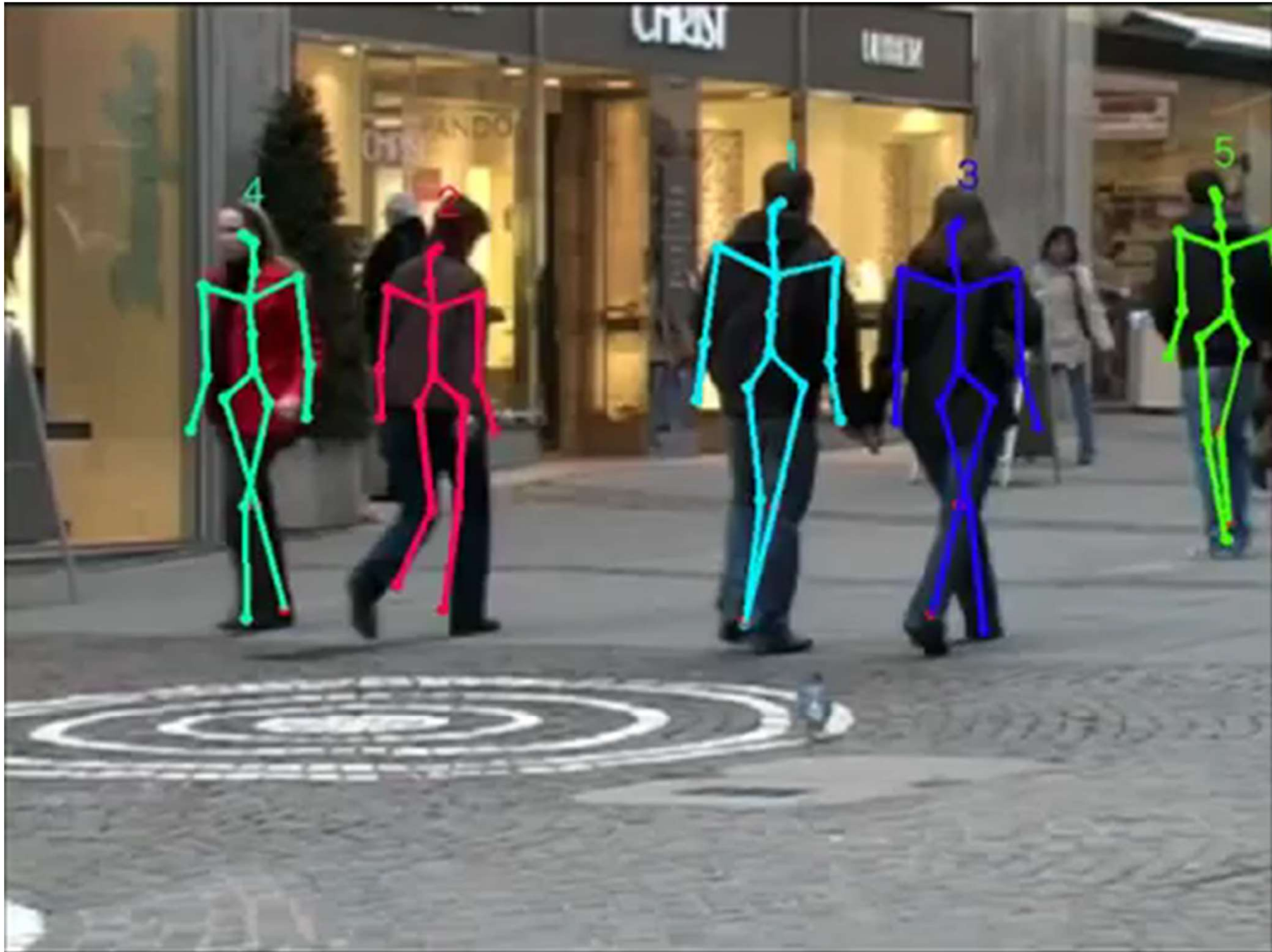
- ***Feature-based methods (e.g. SIFT+Ransac+regression)***
 - Extract visual features (corners, textured areas) and track them over multiple frames
 - Sparse motion fields, but possibly robust tracking
 - Suitable especially when image motion is large (10-s of pixels)
- ***Direct-methods (e.g. optical flow)***
 - Directly recover image motion from spatio-temporal image brightness variations
 - Global motion parameters directly recovered without an intermediate feature motion calculation
 - Dense motion fields, but more sensitive to appearance variations
 - Suitable for video and when image motion is small (< 10 pixels)

Long-term Robust Visual Tracking via Temporal Learning and Deep Neural Networks

Shu Wang

Department of Computer Science, Rutgers University.

More Tracking Demos



More demos

CVPR 2018

Differentiating Objects by Motion:
Joint Detection and Tracking of Small Flying Objects

Submission ID 59

Supplementary material



Siam Mask Object Segmentation Tracking Demo

<https://www.youtube.com/watch?v=EwxyHHo3qI>

Moving objects estimation

Ziang Chen, Shaodi You, Viorela Ila

motion probability map



We still don't touch these areas



Action Points

- This week:
 - Finish assignment 2
 - Assignment 3 from Friday 😊
- Next week (Martin):
 - Global transform and image stitching
 - Basics of Recognition

Disclaimer

- Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.